

一、仿真环境配置与模型导入

1.实验描述

在智能餐饮场景中，机械臂是商服机器人的重要执行器，承担着取餐、送餐等重要任务，有关机械臂的控制以及机械臂与商服机器人的协同工作是本课程要学习的主要内容。

本次实验内容为机械臂仿真环境的配置以及模型导入。仿真环境配置是基于 ROS 系统的相关操作，模型导入则是将设计完成的机械臂三维模型导入到系统中，并且通过专门的软件环境来生成机械臂各关节间的相对关系等。本次实训不在真实环境中直接操作机械臂，而是在仿真环境中搭建机械臂的模型，虽然不是实体，但是模型的相关数据是与机械臂实体完全一致的，后续的其他仿真操作也建立在本次实验的基础之上。

2.实验目标

- (1) 了解机器人模型准备的相关操作；
- (2) 准备开发工具，并在开发目录中创建工作空间及功能包；
- (3) 导入机械臂模型，并完成仿真环境的相关配置。

3.实验准备

硬件准备：个人 PC，商服机器人一台（带连接线及电源线等配件）。

软件准备：参考《商服机器人原理与实践》实训手册，参照实训一“ROS 配置与实操实验指导”操作步骤，分步完成以下软件配置：

一是个人 PC 上安装 Visual Studio Code 编译工具，可基于 VSCode 进行 C++ 编程或 Python 编程。在机器人上已内置了 VSCode 软件，也可直接在机器人上进行编程或工程代码文件的修改、优化。后续实验的.py 文件或.cpp 文件均可基于 VSCode 进行编辑修改；

二是在个人 PC 上进行以太网配置，继而可基于个人 PC 访问商服机器人上的工程文件，进行远程连接和相关操作。

同时，需要先学会终端 Terminal 的操作方法及 ROS 工作空间创建、进入指定文件夹路径等相关操作。

4.实验步骤

步骤 1：模型准备

本次实训中已提供对应的机械臂模型文件，此步骤了解流程和内容即可。

（1）模型设计

机械臂的模型需要通过专业的 3D 机械绘图软件进行设计。SolidWorks 作为常用的机械设计软件，其自身提供了强大的特征建模功能，通过拉伸、旋转、基准面、基准轴等操作，能够精确实现机械臂的结构设计。同时，其草图绘制和编辑功能使得工程师可以精确控制机械臂的每一个细节，包括尺寸标注、几何约束等，甚至能够生成复杂的曲面和曲线，这些功能确保了设计的准确性和高效性，为机械臂的运动轨迹和作业空间设计提供了极大的便利。SolidWorks 还具备装配体功能，工程师可以使用此功能来标注坐标系，生成精确的装配体模型，从而确保机械臂各部件之间的配合精度和整体性能。

本次实训使用的机械臂模型就是通过 SolidWorks 设计出来的，每个连杆及关节都是通过软件绘制并进行装配。关于模型设计的详细操作，这里不做描述，后续的操作过程中，我们直接使用准备好的机械臂模型。

（2）模型导出

模型导出指的是将 SolidWorks 绘制好的机械臂模型通过一定的操作转换成 ROS 中可仿真运行的文件的步骤。SolidWorks 绘制的机械臂模型主要用于工程制图等，不能直接用于在 ROS 中进行仿真。如果想要实现仿真，就需要将模型导出为 ROS 中可调用的格式，即 URDF 文件。

URDF 全称为 Unified Robot Description Format，中文可以翻译为“统一机器人描述格式”。它是一种 XML 格式的机器人描述语言，用于描述机器人的各种部件，例如连杆、传动、关节、传感器等，以及各部件相关的几何、运动学、惯性参数等信息。URDF 在机器人仿真、视觉感知和运动规划等领域都有着广泛的应用，尤其在 ROS 界非常流行。它支持多种关节类型，包括固定关节、浮动关节、旋转关节、滑动关节、万向节等，并且还支持描述多个连杆的机器人。URDF 可以通过 visual 和 collision 两种类型的模型来表示机器人的外形和碰撞形状，并支持多种文件格式，如 STL、DAE、OBJ 等。通过 SolidWorks 设计的机械臂模型就是 STL 格式。

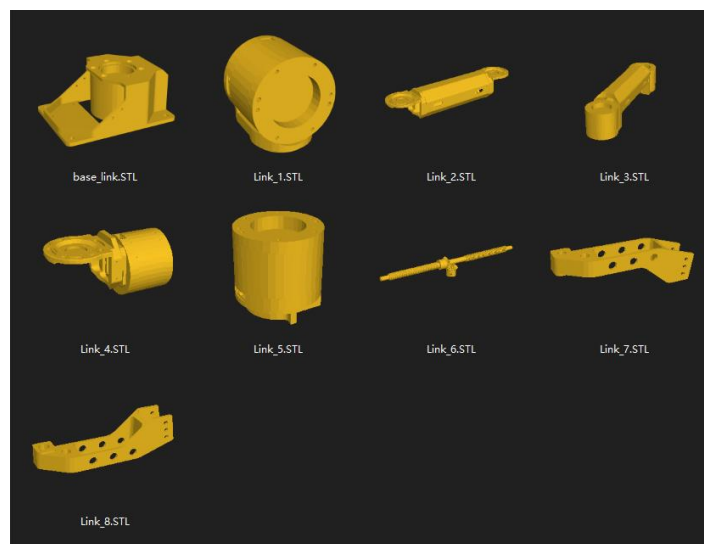


图 1-1 课程提供的机械臂模型文件

在 SolidWorks 中设计完成机械臂模型后，需要通过插件将机械臂模型的三维装配文件转换为可用于仿真的 URDF 文件。实操中使用的插件是 `solidworks_urdf_exporter`。此插件可在 ROS 官网中找到。

ROS.org

About | Support | Discussion Forum | Service Status | Q&A answers.ros.org

Documentation Browse Software News

sw_urdf_exporter

SolidWorks to URDF Exporter

The SolidWorks to URDF exporter is a SolidWorks add-in that allows for the convenient export of SW Parts and Assemblies into a URDF file. The exporter will create a ROS-like package that contains a directory for meshes, textures and robots (urdf files). For single SolidWorks parts, the part exporter will pull the material properties and create a single link in the URDF. For assemblies, the exporter will build the links and create a tree based on the SW assembly hierarchy. The exporter can automatically determine the proper joint type, joint transforms, and axes.

Download Installer

1. !SolidWorks Version Compatibility

There is a known STL export bug with SolidWorks 2018 that exists up to Service Pack 4 that renders this add-in unusable. If you are using 2018, please update to service pack 5 or use SolidWorks 2019 or later. 2017 and below may also work

图 1-2 ROS 插件

根据 SolidWorks 的版本找到对应的插件下载并完成安装后，便可以在 SolidWorks 中打开模型并进行导出，关于导出的详细操作，这里也不做细讲。导出完成后可得到 URDF 文件。将此文件导入到 ROS 中进行使用时，需要根据实际情况进行调整，后面会详细说明模型文件夹中的文件及操作。

步骤 2：开发工具 MoveIt 安装与准备

本次实训中，使用的仿真软件是 MoveIt。

MoveIt 是 ROS 中一系列移动操作的功能包的组成，它包含运动规划、碰撞检测、操作控制、3D 感知、运动学等功能。

在控制机械臂运动方面，MoveIt 能读取机械臂的当前状态，包括关节角度、位置和速度等信息。在接收到规划请求后，MoveIt 会对机械臂的当前状态和任务目标进行运动规划，生成一条可行的轨迹，并选择一条最优的路径来控制机械臂的运动。同时还能实时监控机械臂的运动状态，确保运动的正确执行。

MoveIt 控制机械臂时的主要流程如下：

- 1) 导入模型：根据真实机械臂或者仿真机械臂，创建完整的机器人模型文件，并配置相应的 URDF 文件。
- 2) 模型配置：根据机器人的 URDF 文件，使用 Setup Assistant 工具完成自碰撞矩阵、规划组、终端夹具等配置，并在完成后生成 ROS 功能包。
- 3) 驱动实现：使用 ArbotiX 或者 ros_control 功能包中的控制器插件，实现对机械臂关节的驱动。插件的使用方法一般分为两步：首先创建插件的 YAML 配置文件，然后通过 launch 文件启动插件并加载配置参数。
- 4) 功能控制：MoveIt 提供了 C++、Python、rviz 插件等接口，可以实现机器人关节空间和工作空间下的运动规划，规划过程中会综合考虑场景信息，并实现自主避障的优化控制。

本次实训中，在安装有机臂的商服机器人上，此开发工具已配置完成，可以直接跳过下面的安装步骤，在步骤 5 中可以直接启动。如果想要自行安装开发环境，可以在终端中分别执行以下命令，等待命令完成后即可，如果下载速度较慢，可以切换下载源，具体方法可自行查阅。

Moveit 开发工具安装指令（每一行是一条指令）：

```
sudo apt install ros-melodic-moveit
sudo apt-get install ros-melodic-moveit-visual-tools
sudo apt-get install ros-melodic-franka-description
```

机械臂控制 UI 界面安装指令：

```
sudo apt-get install ros-melodic-joint-state-publisher-gui
```

步骤 3：创建工作空间及功能包

（1）创建和编译工作空间

在 ROS 中，工作空间用于组织和构建 ROS 功能包的目录结构。功能包则是 ROS 的基本组织单元，它包含了一组相关的文件和代码，用于实现特定的功能。

商服机器人本身已经创建了工作空间 workspace，基于商服机器人打开文件目录，在“workspace”路径下创建“sebot_ros_simulation”文件夹。

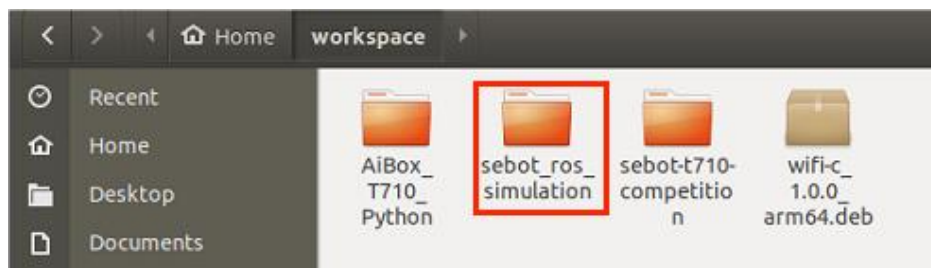


图 1-3 创建 sebot_ros_simulation 文件夹

在“sebot_ros_simulation”文件夹下创建“src”文件夹。

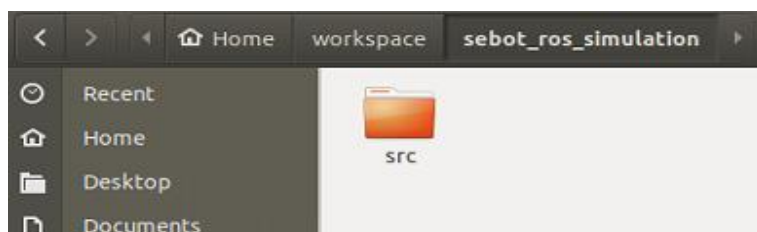


图 1-4 创建 src 文件夹

打开终端，进入到路径 sebot_ros_simulation/src 下，输入以下命令初始化工作空间。

```
catkin_init_workspace
```

```
root@paddlepi:~# cd workspace/sebot_ros_simulation/src/
root@paddlepi:~/workspace/sebot_ros_simulation/src# catkin_init_workspace
Creating symlink "/root/workspace/sebot_ros_simulation/src/CMakeLists.txt" pointing to "/opt/ros/melodic/share/catkin/cmake/toplevel.cmake"
root@paddlepi:~/workspace/sebot_ros_simulation/src#
```

图 1-5 创建工作空间

退回上级目录或者直接进入路径 sebot_ros_simulation 下，输入以下命令编译工作空间。

```
catkin_make
```

```

root@paddlepi:~/workspace/sebot_ros_simulation/src# cd ..
root@paddlepi:~/workspace/sebot_ros_simulation# catkin_make
Base path: /root/workspace/sebot_ros_simulation
Source space: /root/workspace/sebot_ros_simulation/src
Build space: /root/workspace/sebot_ros_simulation/build
Devel space: /root/workspace/sebot_ros_simulation/devel
Install space: /root/workspace/sebot_ros_simulation/install
####
#### Running command: "cmake /root/workspace/sebot_ros_simulation/src -DCATKIN_D
EVEL_PREFIX=/root/workspace/sebot_ros_simulation/devel -DCMAKE_INSTALL_PREFIX=/r
oot/workspace/sebot_ros_simulation/install -G Unix Makefiles" in "/root/workspac
e/sebot_ros_simulation/build"
####
-- The C compiler identification is GNU 7.5.0
-- The CXX compiler identification is GNU 7.5.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped

```

图 1-6 编译工作空间

编译完成后“sebot_ros_simulation”文件夹中会生成另外两个文件夹，“src”文件夹中会生成编译配置文件。

```

Update the VERSION argument <min> value or use a ...<max> suffix to tell
CMake that the project does not need compatibility with older versions.

-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Failed
-- Looking for pthread_create in pthreads
-- Looking for pthread_create in pthreads - not found
-- Looking for pthread_create in pthread
-- Looking for pthread_create in pthread - found
-- Found Threads: TRUE
-- Using Python nosetests: /usr/bin/nosetests-2.7
-- catkin 0.7.29
-- BUILD_SHARED_LIBS is on
-- BUILD_SHARED_LIBS is on
-- Configuring done
-- Generating done
-- Build files have been written to: /root/workspace/sebot_ros_simulation/build
####
#### Running command: "make -j8 -l8" in "/root/workspace/sebot_ros_simulation/bu
ild"
####
root@paddlepi:~/workspace/sebot_ros_simulation#

```

图 1-7 编译完成

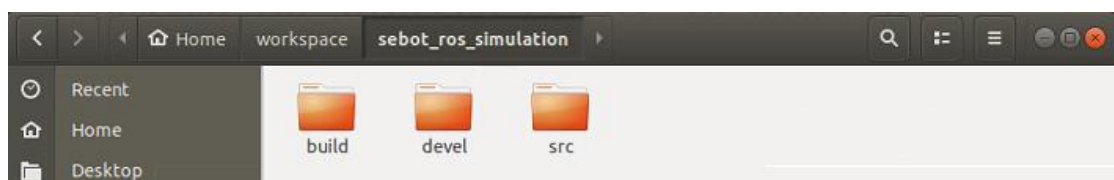


图 1-8 生成的相关文件夹

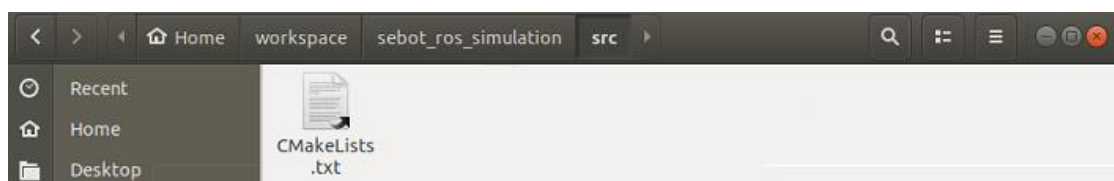


图 1-9 生成的编译文件

(2) 创建功能包

进入到路径 `sebot_ros_simulation/src` 下，输入以下命令创建机械臂仿真的功能包。

```
catkin_create_pkg sebot_arm urdf xacro
```

```
root@paddlepi:~/workspace/sebot_ros_simulation# cd src/
root@paddlepi:~/workspace/sebot_ros_simulation/src# catkin_create_pkg sebot_arm
urdf xacro
Created file sebot_arm/package.xml
Created file sebot_arm/CMakeLists.txt
Successfully created files in /root/workspace/sebot_ros_simulation/src/sebot_arm
. Please adjust the values in package.xml.
root@paddlepi:~/workspace/sebot_ros_simulation/src#
```

图 1-10 创建功能包

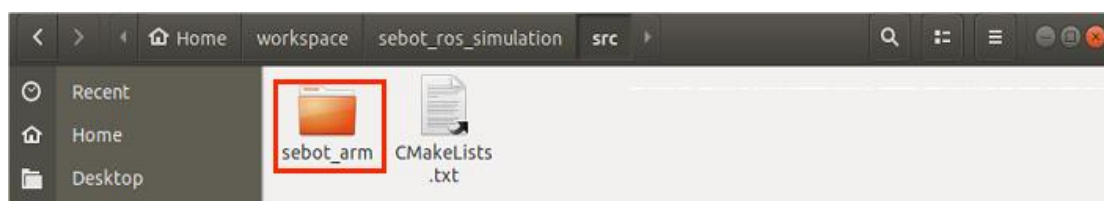


图 1-11 生成的功能包



图 1-12 功能包内的相关文件

步骤 4：模型导入并查看

此步骤基于已经配置完成的文件，目的是了解最终的效果。在下一步骤中将详细说明配置的逻辑过程和操作步骤。

(1) 模型导入并重新编译

通过远程共享文件的方式（参考商服机器人的远程共享文件操作），将课程提供的实训 1 中“sebot_arm”文件夹下的“urdf”、“meshes”、“launch”、“config”四个文件夹拷贝到商服机器人的 `sebot_ros_simulation/src/sebot_arm` 路径下。

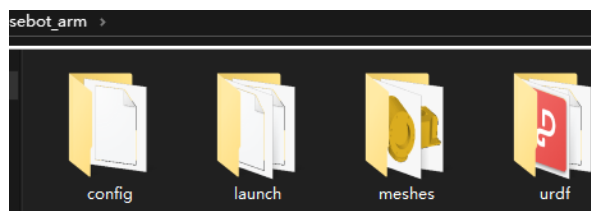


图 1-13 模型配置文件

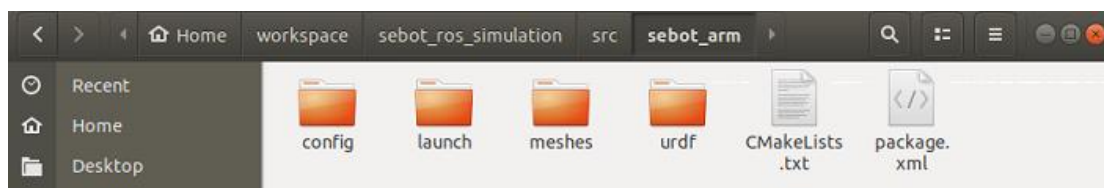


图 1-14 拷贝完成的模型配置文件

这四个文件夹包含了机械臂仿真所需的相关模型、配置文件、启动文件等。“urdf”文件夹下存放的是关于机器人模型相关参数的 URDF 或 xacro 文件；“meshes”文件夹中存放的是机械臂的模型渲染文件；“launch”文件夹中存放的是运行功能所需的相关启动文件；“config”文件夹存放的是 Rviz 的配置文件。

文件拷贝完成后，在终端中进入路径 `sebot_ros_simulation`，重新编译整个工作空间。编译命令如下：

```
catkin_make
```

```

root@paddlepi: ~/workspace/sebot_ros_simulation
File Edit View Search Terminal Help
Update the VERSION argument <min> value or use a ...<max> suffix to tell
CMake that the project does not need compatibility with older versions.

-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
-- Using Python nosetests: /usr/bin/nosetests-2.7
-- catkin 0.7.29
-- BUILD_SHARED_LIBS is on
-- BUILD_SHARED_LIBS is on
-- traversing 1 packages in topological order:
--   - sebot_arm
-- ++ processing catkin package: 'sebot_arm'
-- ==> add_subdirectory(sebot_arm)
-- Configuring done
-- Generating done
-- Build files have been written to: /root/workspace/sebot_ros_simulation/build
####
#### Running command: "make -j8 -l8" in "/root/workspace/sebot_ros_simulation/build"
####
Built target sebot_arm_xacro_generated_to_devel_space_
root@paddlepi:~/workspace/sebot_ros_simulation#

```

图 1-15 编译工作空间

(2) 通过 Rviz 查看模型

打开终端，进入到工作空间的路径 `sebot_ros_simulation` 下，输入如下指令：

```
source devel/setup.bash
```

在 ROS 中，`source` 命令用于加载环境变量和配置文件。当在终端中执行 `source` 命令时，系统会从指定的文件中读取并执行其中的命令，从而将文件中定义的环境变量和配置加载到当前的 shell 环境中。`source` 命令常用于加载 ROS 的工作空间配置，尤其是 `setup.bash` 或 `setup.zsh` 等文件。这些配置文件会设置 ROS 所需的环境变量，包括 ROS 主目录、ROS 工作空间、ROS 软件包路径等。通过执行 `source` 命令，可以确保当前终端会

话具备 ROS 所需的正确环境，从而能够顺利运行 ROS 相关的命令和程序。如 `roscore`、`roslaunch`、`roslaunch`、`roslaunch` 等。

特别注意：在后续的仿真操作中，每需要新开一个终端并且运行 ROS 命令时，都需要先执行此步骤，否则可能无法正确执行对应的 ROS 命令。已经执行过此命令的终端无需重复此步骤。

```
root@paddlepi:~# cd workspace/sebot_ros_simulation/
root@paddlepi:~/workspace/sebot_ros_simulation# source devel/setup.bash
root@paddlepi:~/workspace/sebot_ros_simulation#
```

图 1-16 执行 source 命令

接着输入以下指令，启动 Rviz。Rviz 中可以查看机械臂的模型。RViz 是 ROS 官方提供的一款 3D 可视化工具，可以将机器人的相关数据进行图形化显示。

```
roslaunch sebot_arm display.launch
```

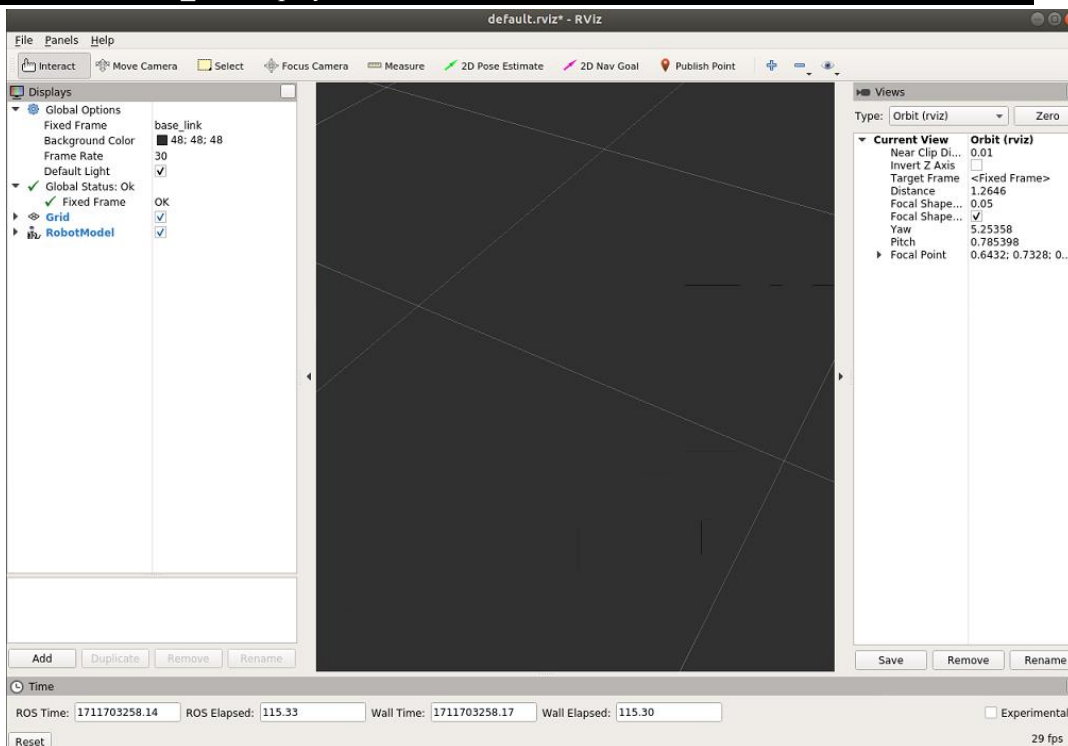


图 1-17 Rviz 软件界面

在 Rviz 左侧菜单栏中点击 “Add”，在弹出的窗口中选择 “RobotModel” 并确认，此步骤用于加载机器人模型。

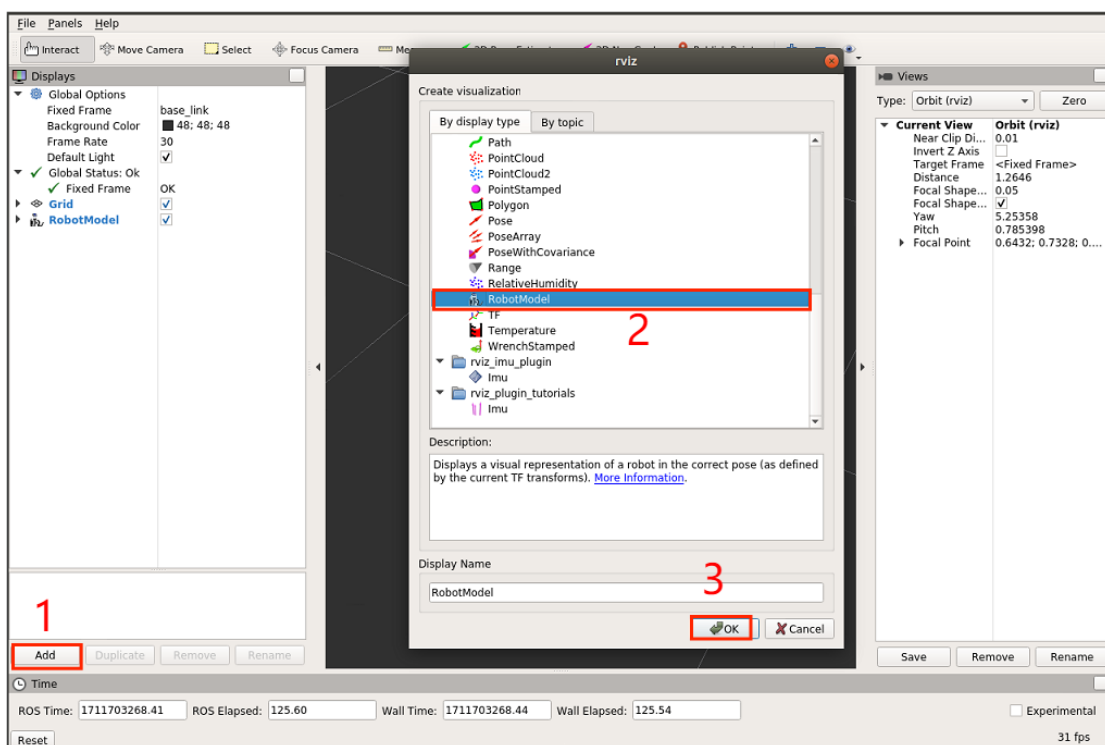


图 1-18 加载机器人模型

在左侧窗口的“Fixed Frame”处改为“base_link”，此步骤用于固定参考坐标系，坐标系确定后才能查看空间中的机器人模型。

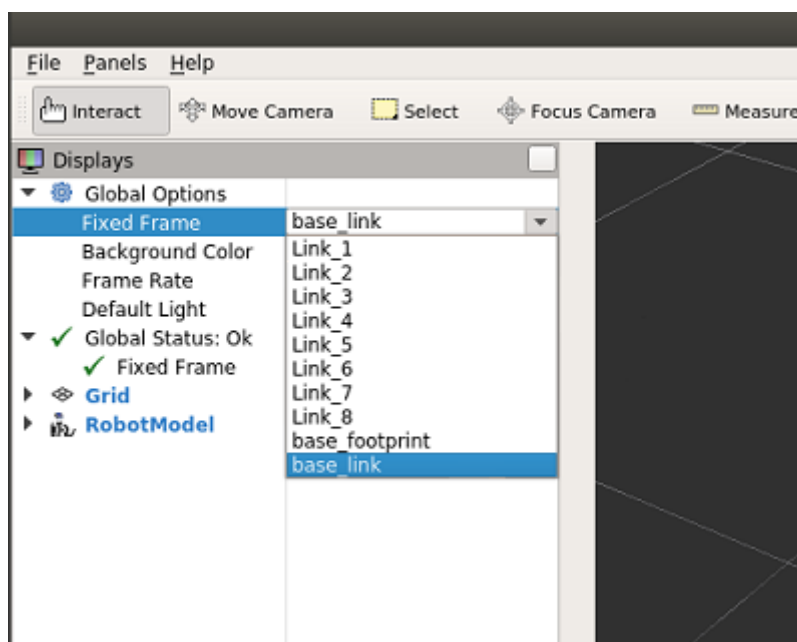


图 1-19 设置固定坐标系

为了便于下次直接查看，需要保存相关配置。单击顶端的菜单栏“File”，选择“Save Config”，相关设置即保存成功。

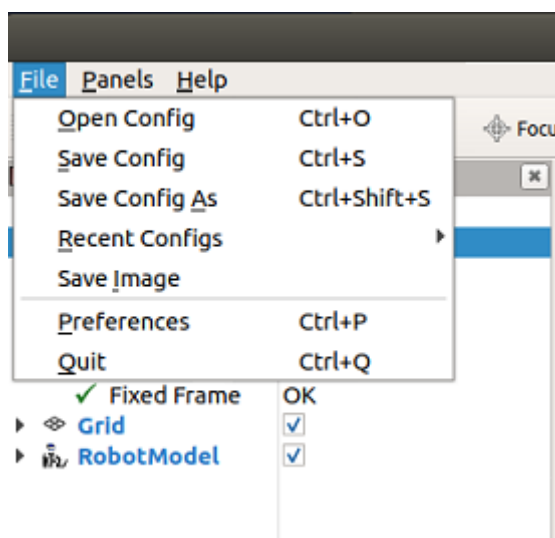


图 1-20 保存设置

全部操作完成后，即可在 Rviz 中查看机械臂的 3D 模型。在操作窗口中，按住右键向上移动鼠标可以放大视图，向下移动鼠标可以缩小视图，按住左键调整视角，按住滚轮可平移视角。

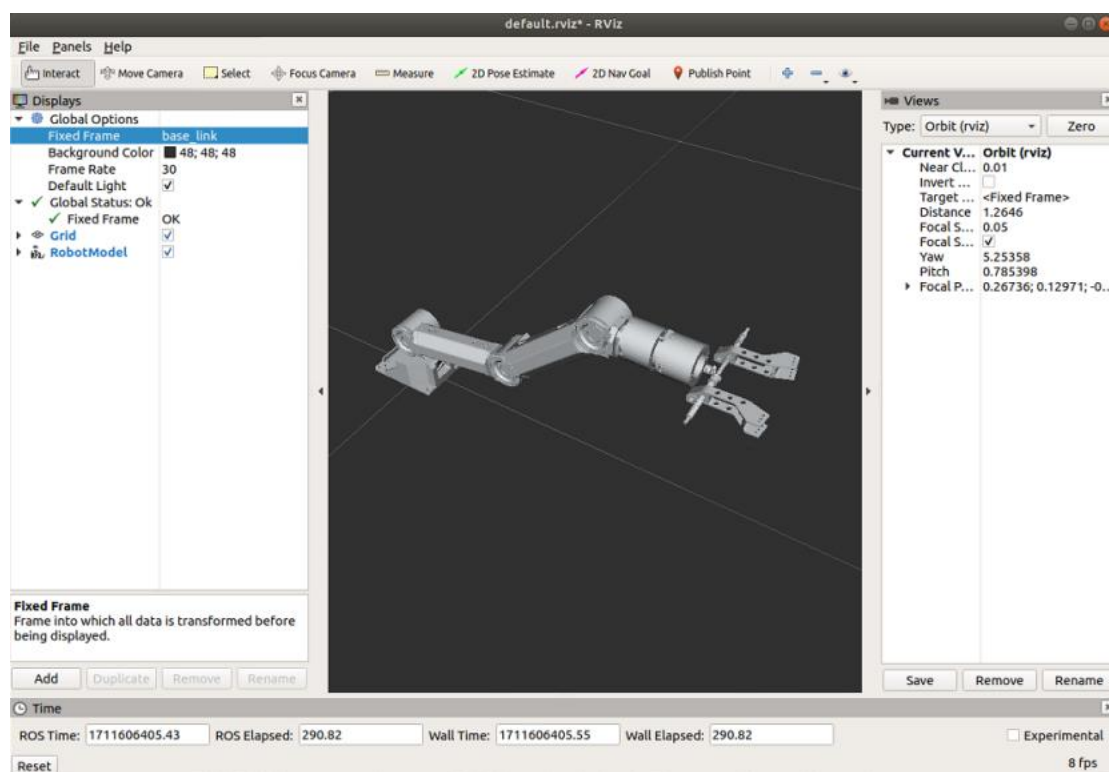


图 1-21 查看机械臂模型

如果想要通过 TF 坐标的形式查看机械臂的联系，可以重新打开一个新终端，进入到 `sebot_ros_simulation` 路径内，先使用 `source` 命令加载环境变量。然后进入到 `src/sebot_arm/urdf/` 路径内再输入以下命令。此命令将在 `urdf` 文件夹下，生成模型 TF 坐标的 pdf 文档。打开文档即可查看模型的 TF 坐标联系。此步骤可直接略过，若想查看 TF 坐

标可自行执行命令。

```
urdf_to_graphviz sebot_arm.urdf
```

```
root@paddlepi:~# cd workspace/sebot_ros_simulation/src/sebot_arm/urdf/
root@paddlepi:~/workspace/sebot_ros_simulation/src/sebot_arm/urdf# urdf_to_graph
viz sebot_arm.urdf
Created file sebot_arm.gv
Created file sebot_arm.pdf
root@paddlepi:~/workspace/sebot_ros_simulation/src/sebot_arm/urdf#
```

图 1-22 生成文档命令



图 1-23 生成的 PDF 文档

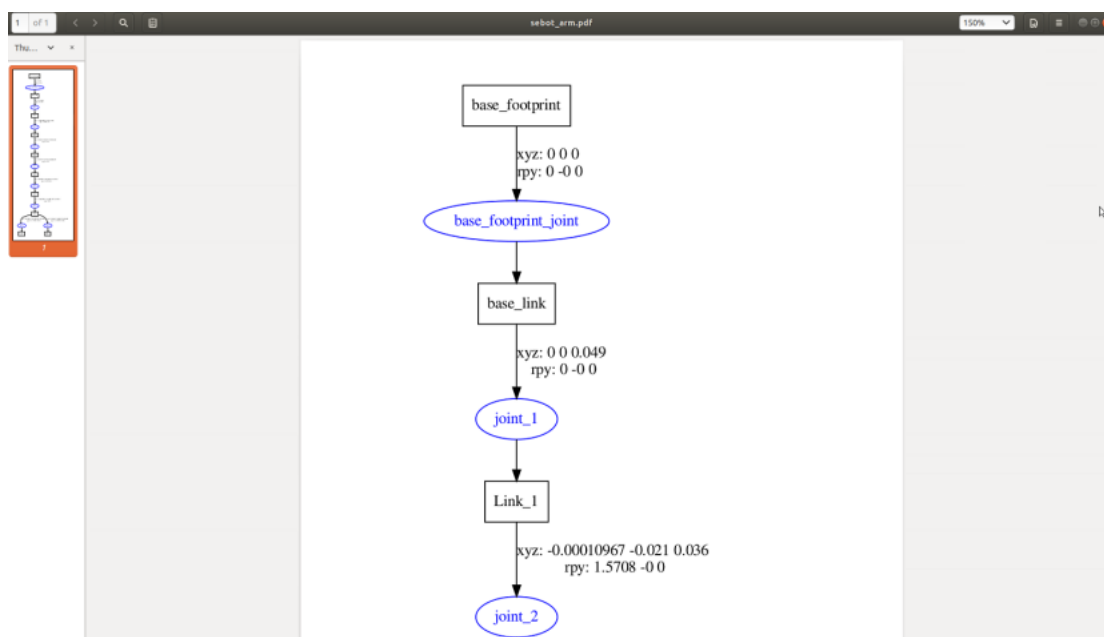


图 1-24 文档内容

步骤 5：配置机械臂

上述步骤是查看已配置完成的机械臂模型，而此步骤的学习内容是为了掌握机械臂的配置过程，最终得到的效果实际上与前一步骤相同，但是可以理解其详细的配置方法并且进行自主开发。

本次实训中配置机械臂使用的是 Setup Assistant。在 ROS 中，Setup Assistant 可以根据用户导入的机器人 URDF 文件生成 SRDF 文件，从而创建一个 MoveIt 配置的功能包。这有助于完成机器人的配置、可视化和仿真等工作。

通过 Setup Assistant，用户可以执行一系列可视化配置任务，如加载机器人 URDF 模型、配置自碰撞矩阵、绑定虚拟关节、创建规划组、定义机器人位姿、配置末端执行器、配置被动关节、设置作者信息以及生成配置文件等。这些步骤对于机械臂在仿真环境中的正确运行至关重要。下面进行相关配置操作。

(1) 加载 URDF

首先关闭前一步骤中的 Rviz 软件，防止对本步骤造成干扰。

在终端中进入到 `sebot_ros_simulation` 路径内，先执行 `source` 命令，接着输入以下命令即可启动 Setup Assistant。

```
roslaunch moveit_setup_assistant setup_assistant.launch
```

Setup Assistant 启动后，选择“Create New MoveIt Configuration Package”按键以创建 SRDF 文件。

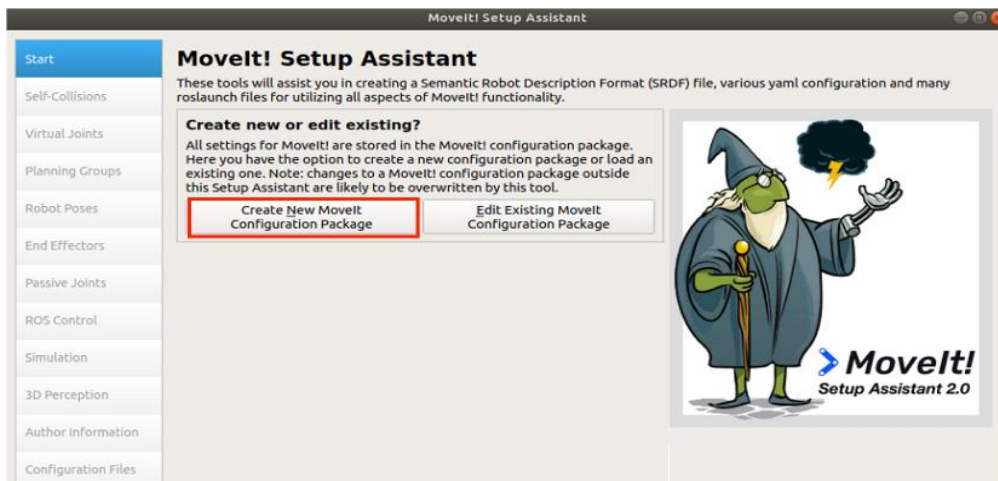


图 1-25 启动 MoveIt

在弹出的窗口中点击“Browse”按键，选择功能包下的机械臂文件（.urdf 文件），文件的路径为 `/workspace/sebot_ros_simulation/src/sebot_arm/urdf/sebot_arm.urdf`。

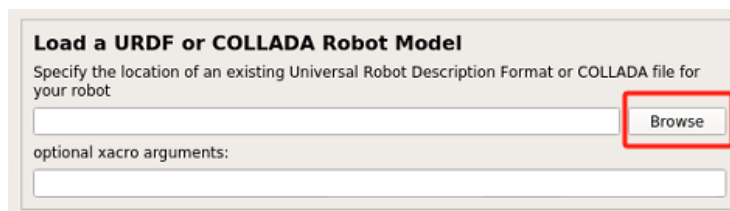


图 1-26 选择 URDF 文件路径

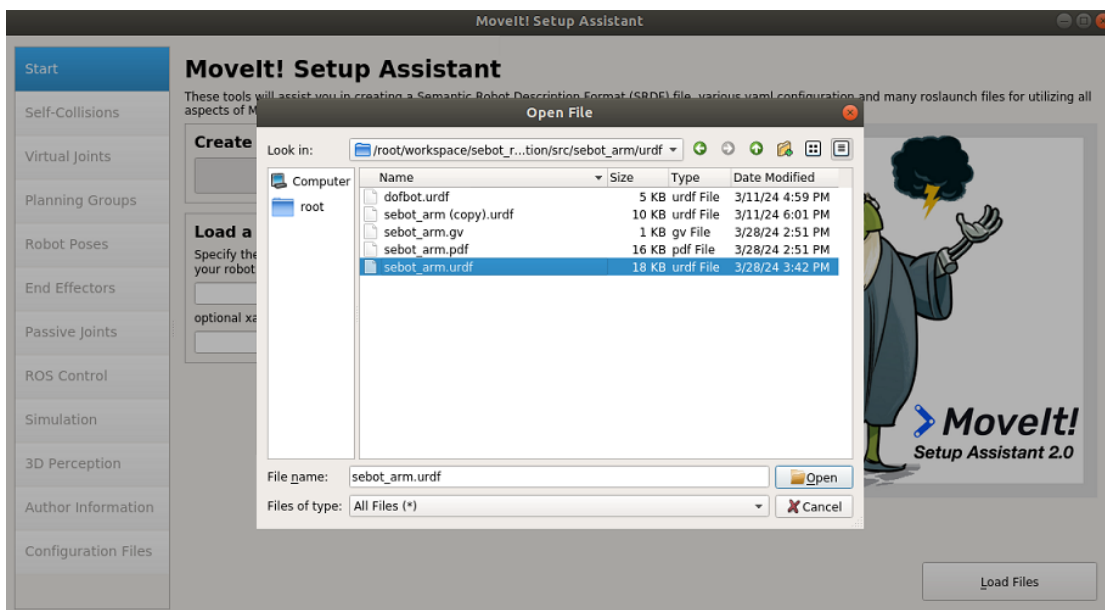


图 1-27 选择 URDF 文件

单击“Open”确认文件后，再点击“Load Files”按键，导入模型，中间如果出现弹窗直接单击确认即可。

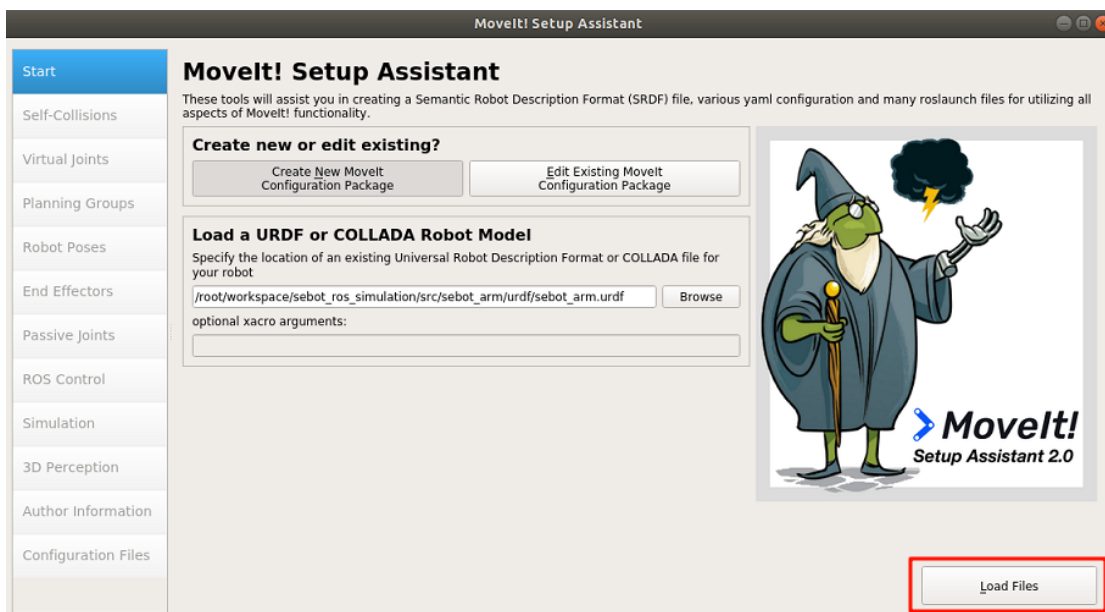


图 1-28 加载 URDF 文件

导入成功后可在软件右侧显示机械臂模型的三维视图，操作方式与 Rviz 中一致，对应的窗口也可以通过拖拽实现大小调整。

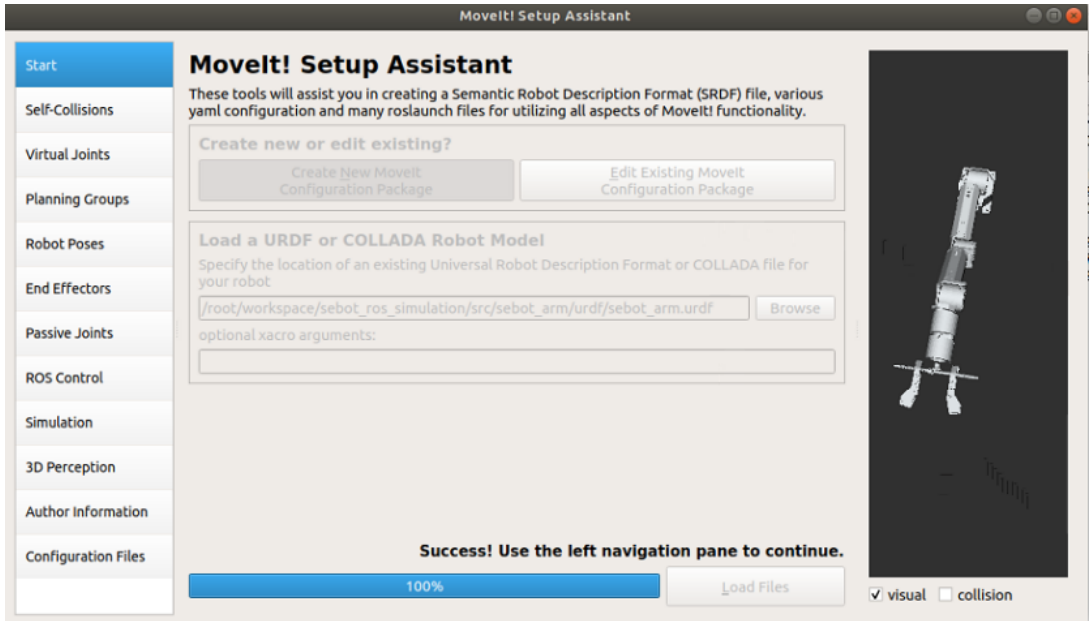


图 1-29 加载完成

(2) 自碰撞矩阵配置

在 MoveIt! Setup Assistant 中点击导航栏左侧的第二项配置步骤，配置自碰撞矩阵。MoveIt 可以设置一定数量的随机采样点，根据这些点生成配装参数，过多的点会造成运算速度慢，过少的点会导致参数不完善，默认的采样点数量是 10000 个。要获得不错的效果，10000 是最低要求，按照这个默认值生成碰撞矩阵。

此步骤会影响到后续仿真中机械臂的自碰撞检测（即自身各部件间的碰撞检测），使用默认的 10000 数值即可，操作时直接单击“Generate Collision Matrix”按键完成配置即可。

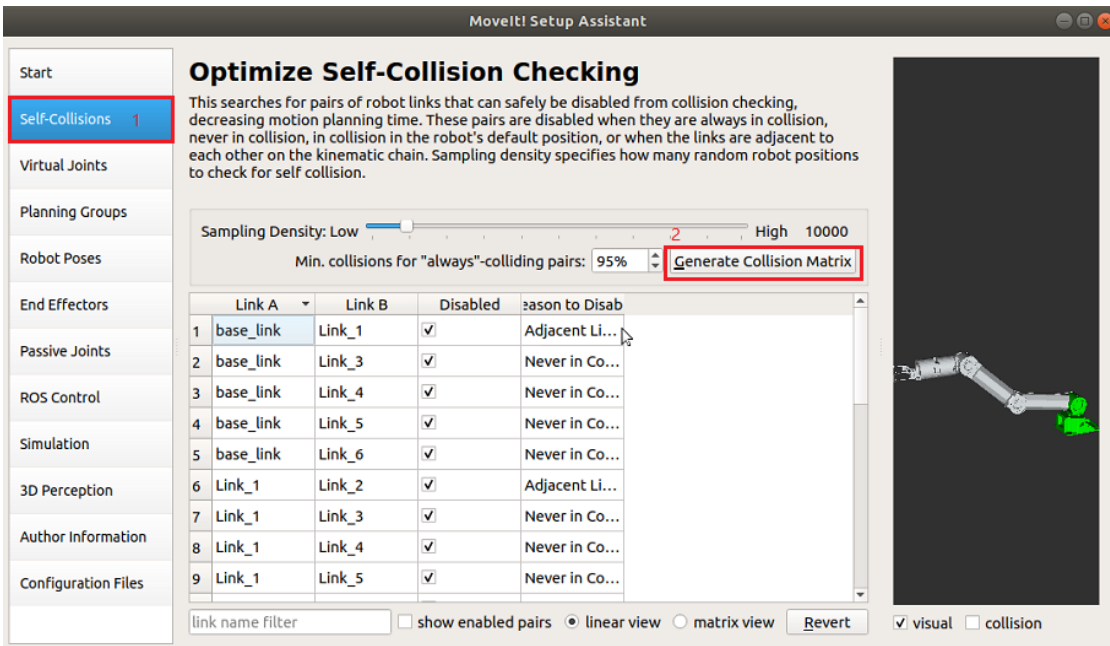


图 1-30 生成自碰撞矩阵

(3) 绑定虚拟关节

绑定虚拟关节主要是用来描述机器人在固定参考坐标系下的位置，如果机器人是移动的，虚拟关节可以与移动基座关联，不过一般的机械臂都是固定不动的，所以也可以不需要虚拟关节。

此步骤中点击 “Add Virtual Joints” 进行虚拟关节添加。

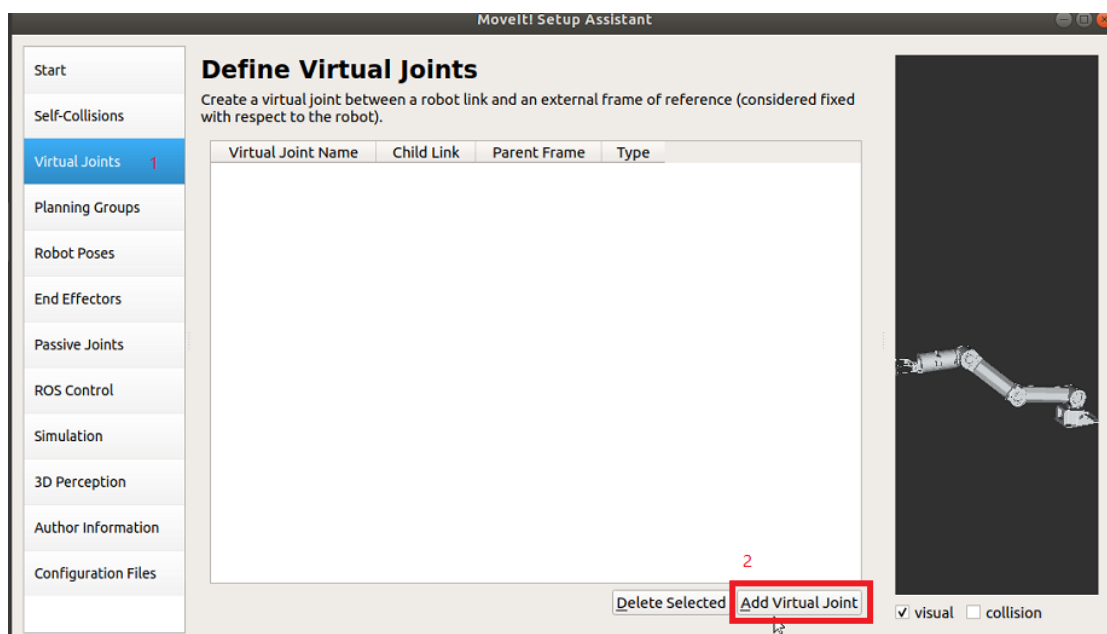


图 1-31 添加虚拟关节

“Virtual Joint Name” 为虚拟关节名称，命名为 “virtual_joint” 即可；“Child Link” 为子链接，选择 “base_link” 即可；“Parent Frame Name” 为世界坐标的名称，命名为 “world” 即可；“Joint Type” 为关节类型，选择 “fixed” 即可。

完成后在界面右下角点击 “Save” 保存。

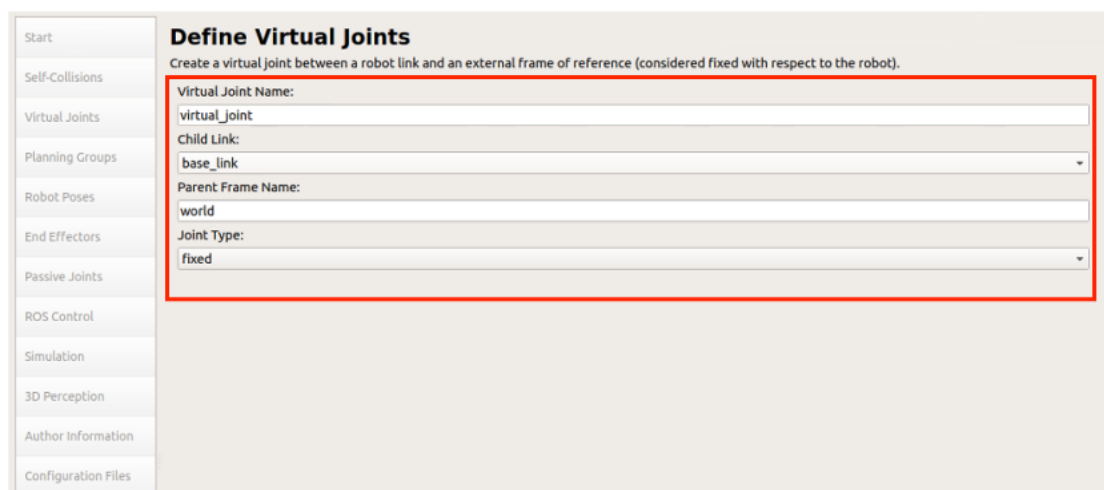


图 1-32 定义虚拟关节

（4）创建规划组

创建规划组指的是将机器人的多个关节集成到一个组当中，运动规划会针对选定的规划组完成路径计算。配置运动组时可以选择运动学解析器（kinematic solver）。

操作时先点击“Add Group”按键：

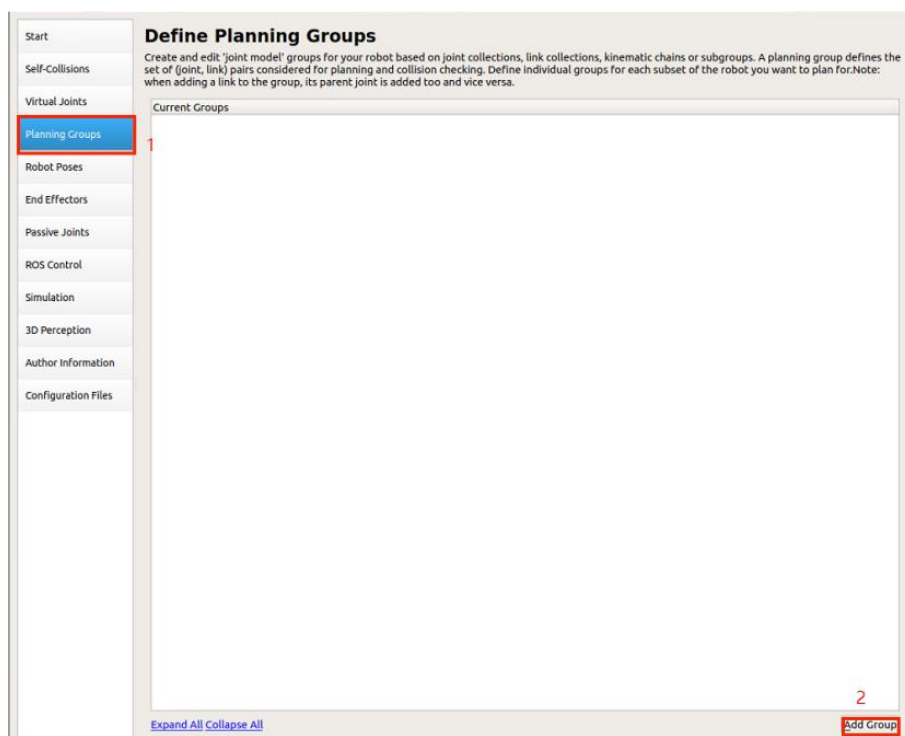


图 1-33 添加规划组

在弹出的窗口中创建机械臂躯干（arm）组；“Kinematic Solver”为正逆运动学求解器，这里选择“kdl”即可；接下来是运动学求解所需的时间和次数，这里使用默认的0.005和3即可；“OMPL Planning”为路径规划算法，这里选择“RRTConnect”，此方法的计算速度更快；最后点击“Add Kin.Chain”按键添加运动学求解链。

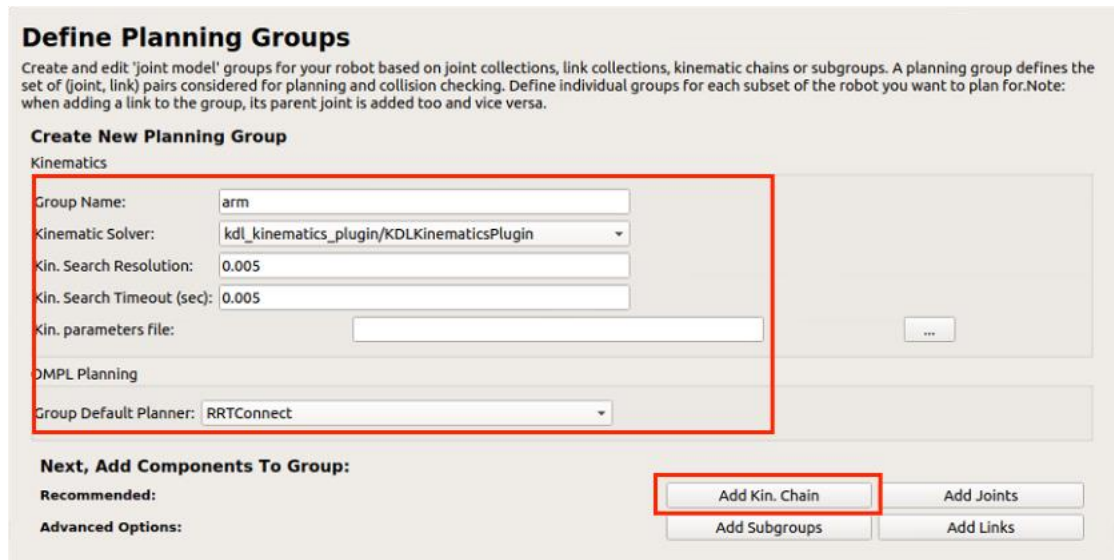


图 1-34 设置规划组

在弹出的窗口中，选择链接。按照下图标识的顺序依次选择，最后点击“Save”保存。

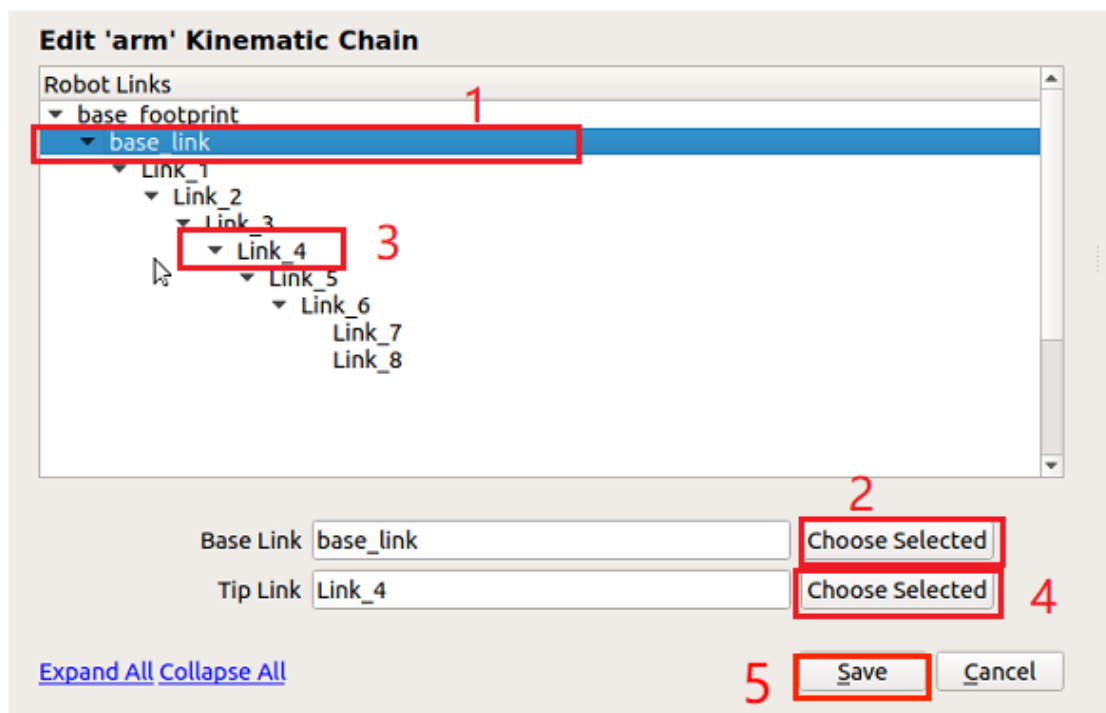


图 1-35 选择链接

保存后即可生成机械臂躯干（arm）的动作组，按照上述步骤再创建夹具（gripper）的动作组。

最终在主界面中可以看到创建好的动作组，机械臂躯干（arm）和夹具（gripper）的运动组构建结果如下。

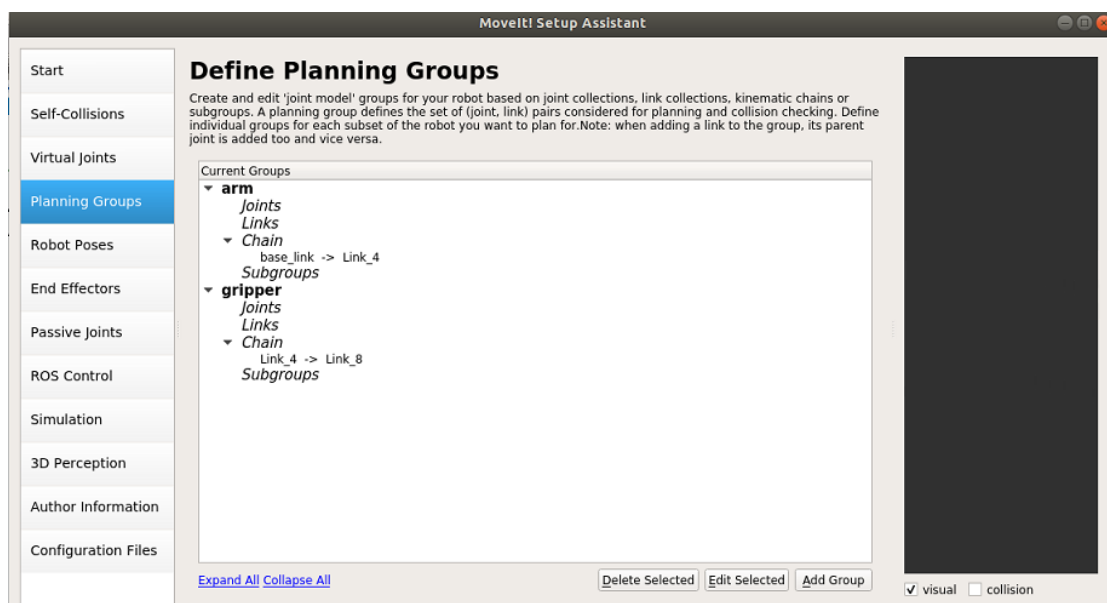


图 1-36 创建完成的规划组

(5) 创建预置机器人位姿

此操作用于设置一些机械臂预设的位姿（即整体的动作），从而在控制时一键执行相关动作，比如复位，直立，抓取物资等。

点击 “Add Pose” 按键进行预置位姿添加。

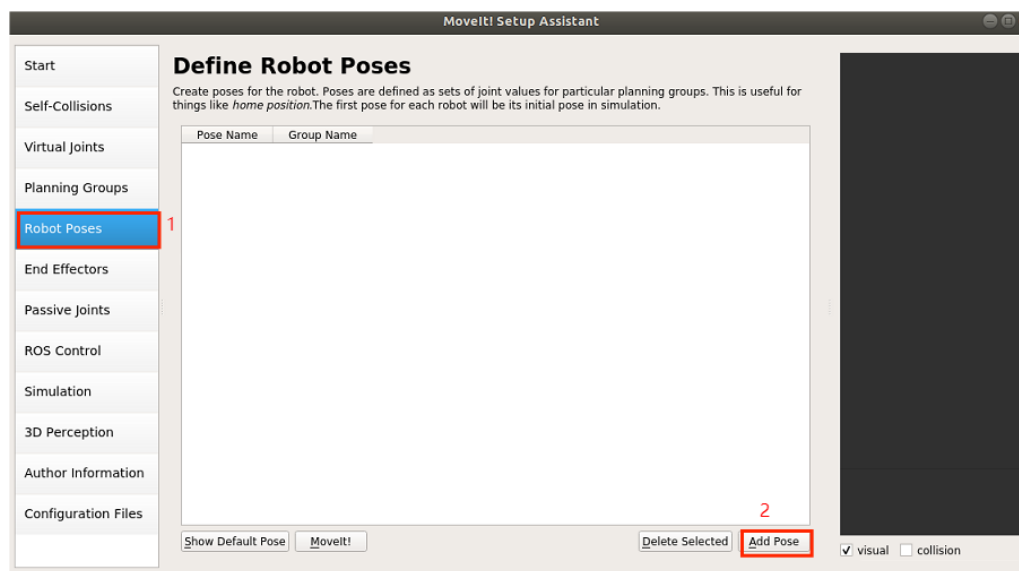


图 1-37 添加预置位姿

弹出的窗口中分别填写 “Pose Name”（动作名称），选择 “Planning Group”（运动组），调整各个 “joint”（关节数值），设置关节数值时可在右侧窗口中看到当前机械臂的动作。完成动作设置后点击 “Save” 可保存当前动作。

此步骤会影响后续的路径规划操作，可以适当多建立几个动作以备后续使用。

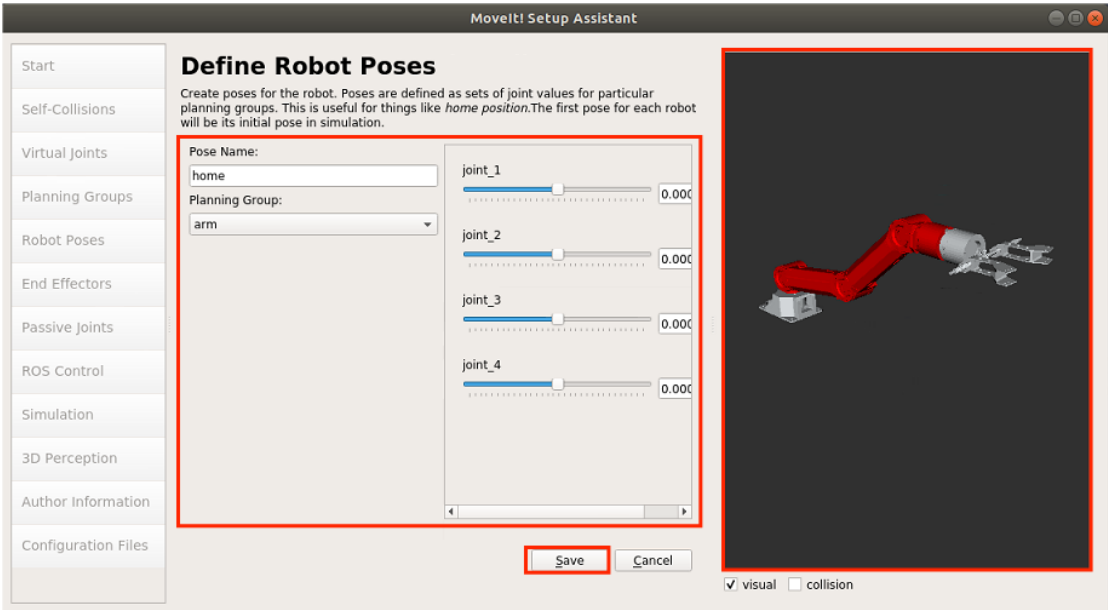


图 1-38 设置关节参数

创建完成的动作会保存在主界面中。

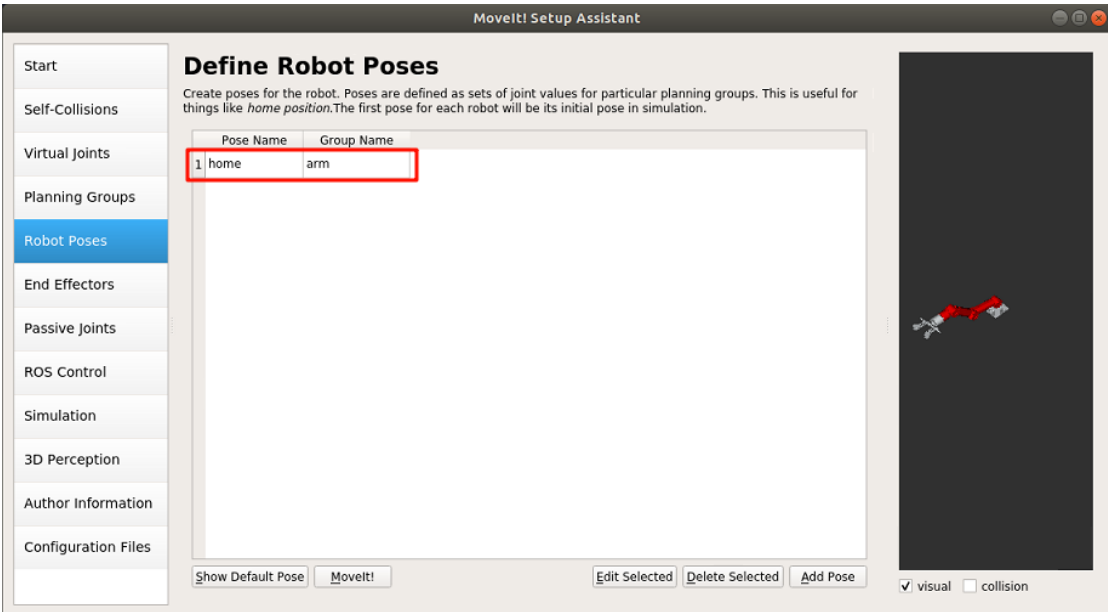


图 1-39 设置完成的位姿

(6) 配置末端执行器

前一步骤中设置的是机器人整体的预置位姿，主要控制的是起始关节与末端执行器中间的关节，而本功能配置的是末端执行器的位姿。此功能的目的是为了独立配置最终的执行器，以此方便任务的实现。

在机械臂中，此功能配置的即终端的夹具。单击“Add End Effector”按键添加夹具。

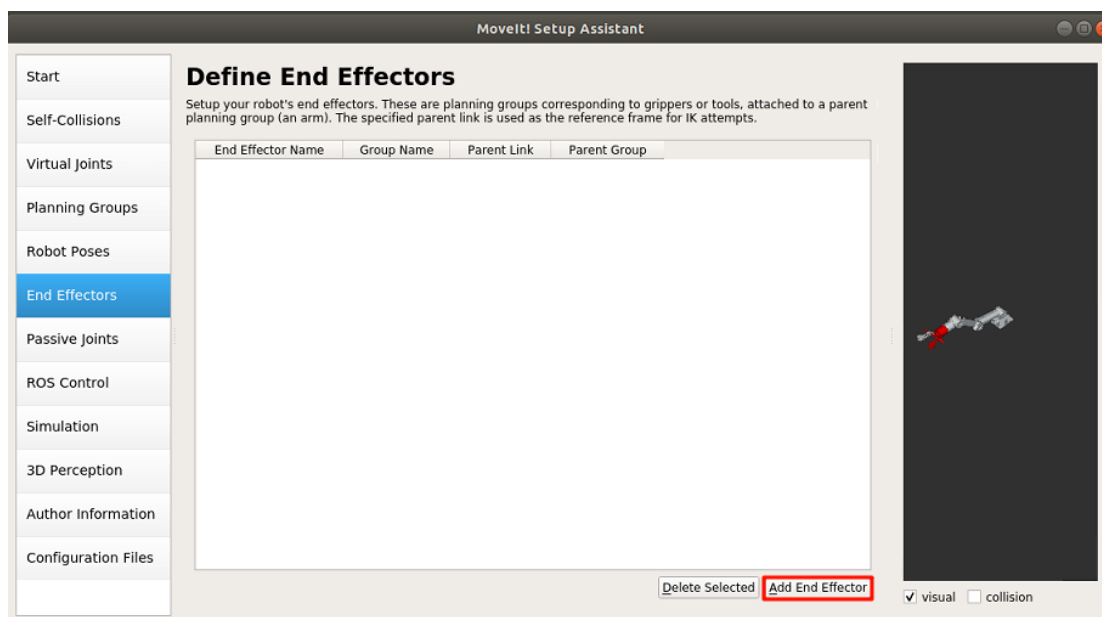


图 1-40 添加末端执行器

在弹出的窗口中，按照下图进行配置。“End Effector Name”表示夹具名称，命名为“gripper”即可；“End Effector Group”表示夹具动作组，选择 gripper 即可；“Parent Link (usually part of the arm)”表示父链接，即夹具前面连接的机械臂节点，选择“Link_4”即可；“Parent Group (optional)”表示父动作组，选择“arm”即可。

创建完成后单击“Save”按钮保存。

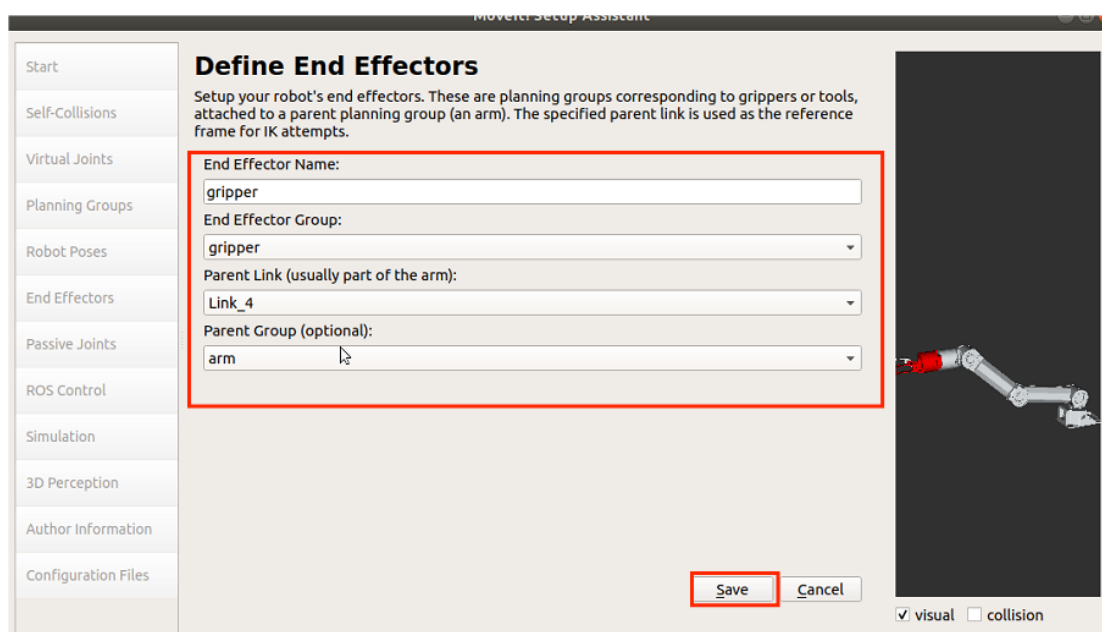


图 1-41 设置夹爪

(7) 添加被动关节

被动关节是指机器人中存在的不需要驱动关节。当某关节被指定为被动关节后，规划器就不会对这个关节的运动进行规划和控制了。本次实训中无需操作此功能，直接跳过即可。

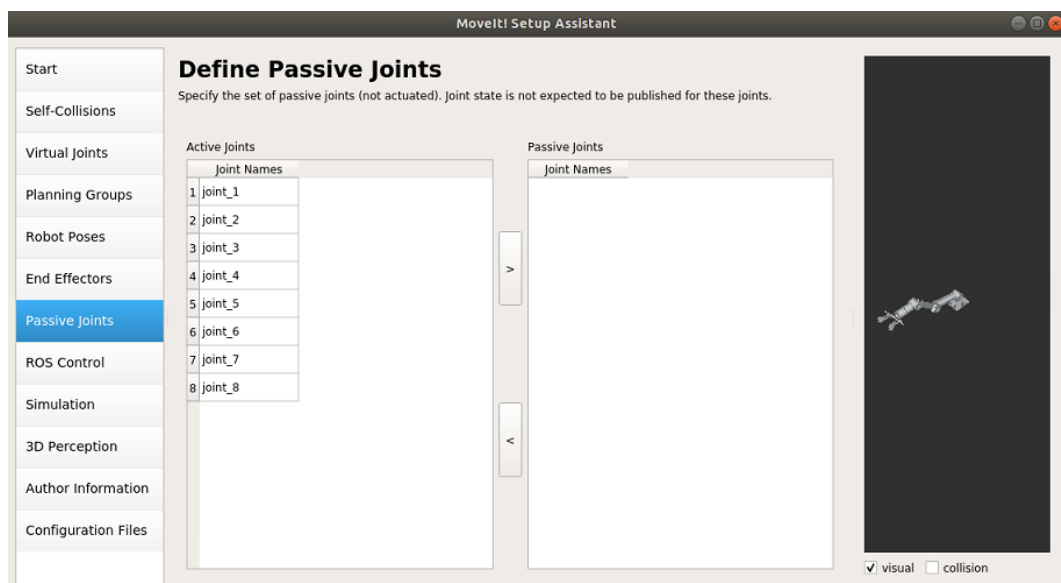


图 1-42 设置被动关节

(8) ROS 控制

ROS 控制是一系列十分常用的与机器人控制相关的功能包，此功能可以生成用于驱动机器人的仿真控制器。通过多个接口可以实现与 Gazebo 或机器人本体的通信，这将帮助我们建立 ROS 和 Moveit 的联系，从而更方便的进行机械臂的控制。

使用时直接点击 “Auto Add FollowJointTrajectory” 按键等待自动生成即可。

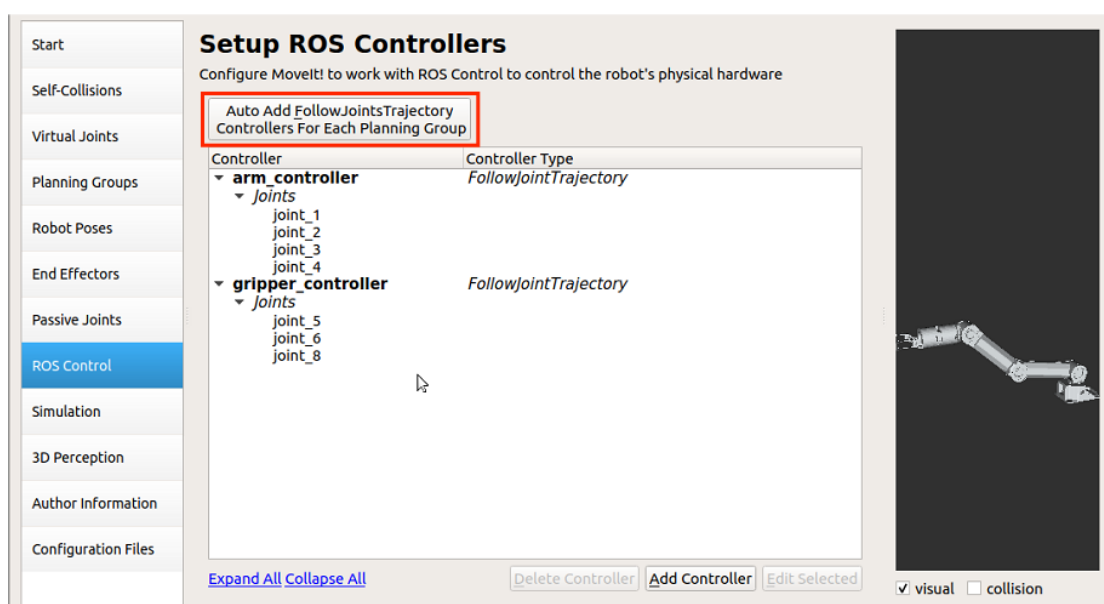


图 1-43 生成 ROS 控制

(9) 生成仿真模型

仿真功能可以帮助我们在 Gazebo 中进行仿真。在此可以生成一个兼容 Gazebo 的新 URDF 文件，此文件是仿真必备文件。

此步骤可以直接跳过。如果需要自行开发，则点击“Generate URDF”创建新的 URDF 文件，此步骤将生成对应的参数配置代码；接着将生成的代码覆盖到“sebot_arm.urdf”文件中。注意将代码中的“EffortJointInterface”改成“PositionJointInterface”。

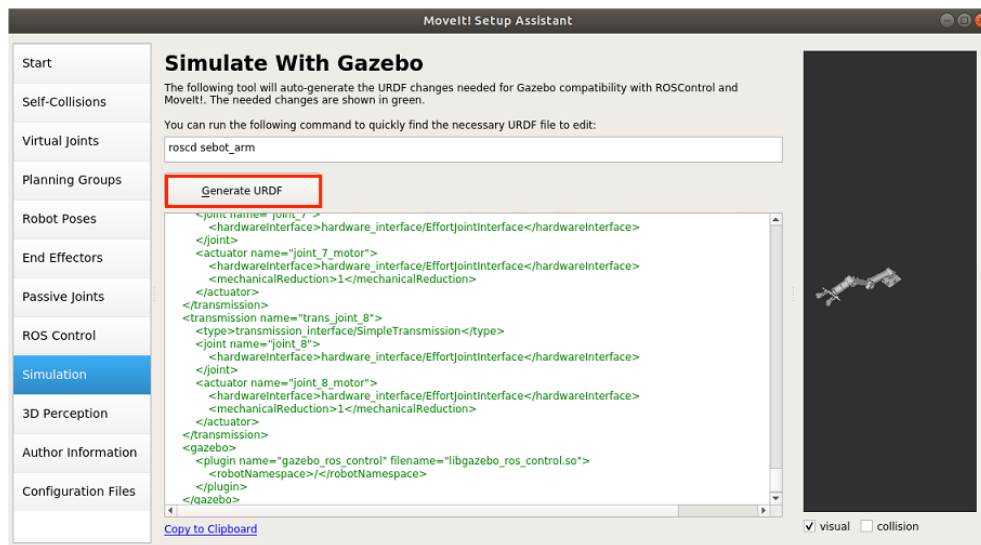


图 1-44 生成 URDF 文件

(10) 配置 3D 感知

3D 感知功能可以在对应的配置文件中配置传感器相关的参数，如果机器人的结构中不包含有传感器并且需要在仿真中使用传感器的功能，那么就需要配置此文件。

在本实验中，此步骤无需配置，直接跳过即可。

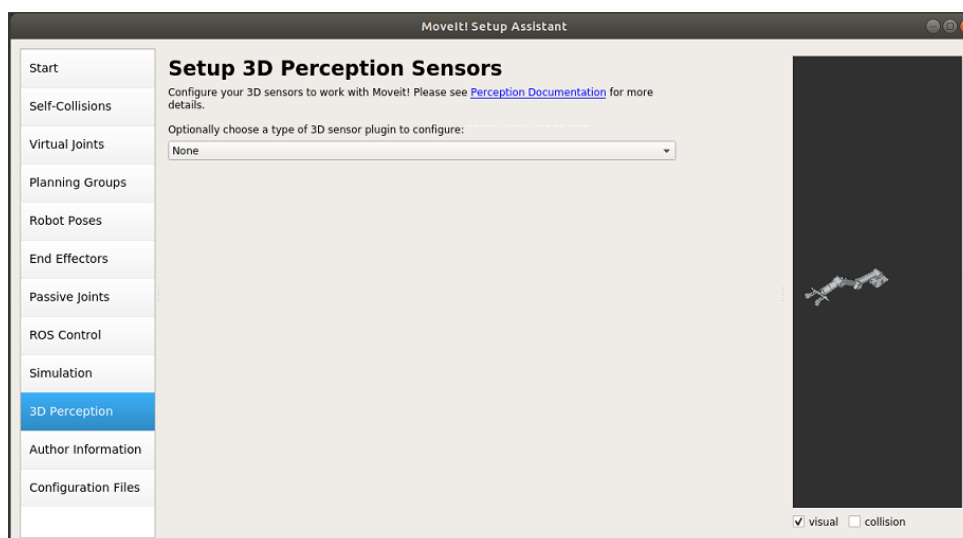


图 1-45 配置 3D 感知

(11) 编辑作者信息

此功能不参与机械臂的控制，其目的是用于添加作者信息，如果将来发布功能包，那么可以通过此信息找到作者。

此步骤中作者信息随意填写即可，不填写无法生成功能包。

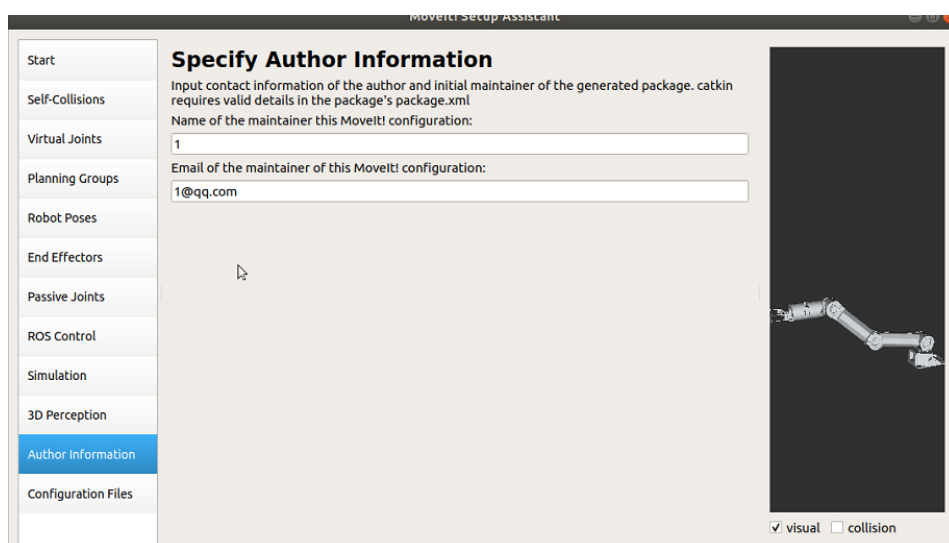


图 1-46 编写作者信息

(12) 生成配置文件

最后一步是生成配置文件，完成功能包的创建。前面的所有配置操作都可以通过此功能生成对应的文件，这样可以将配置信息保存并方便后续的使用。

操作时先单击“Browse”选择文件保存的路径，然后单击“Generate Package”按键生成配置文件。如果配置文件已有，程序会出现弹窗提示，如果没有配置文件，则会自动在所选文件夹下生成对应的内容。

例如：在工作空间 `sebot_ros_kits/src` 下新建一个文件夹“`sabot_arm_moveit_config`”。并选择该目录作为保存路径，接着单击“Generate Package”，进度条完成后即配置完成，接着单击“Exit Setup Assistant”即可退出此可视化配置工具。

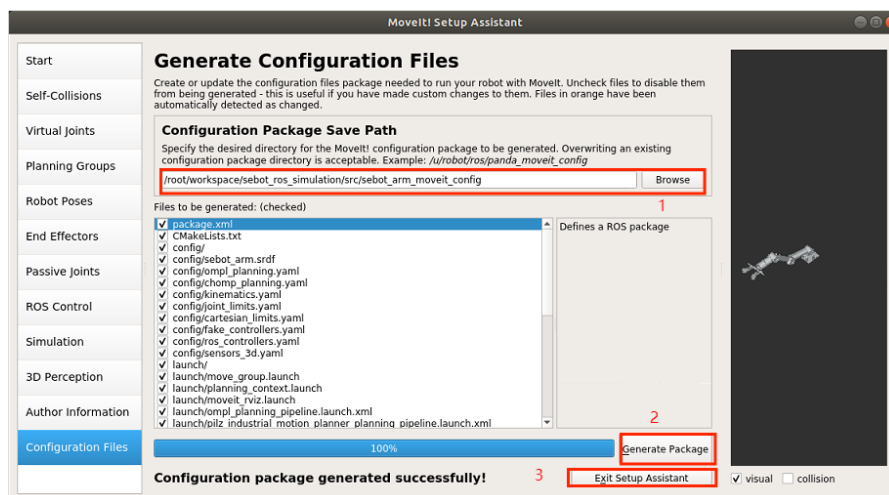


图 1-47 生成配置文件

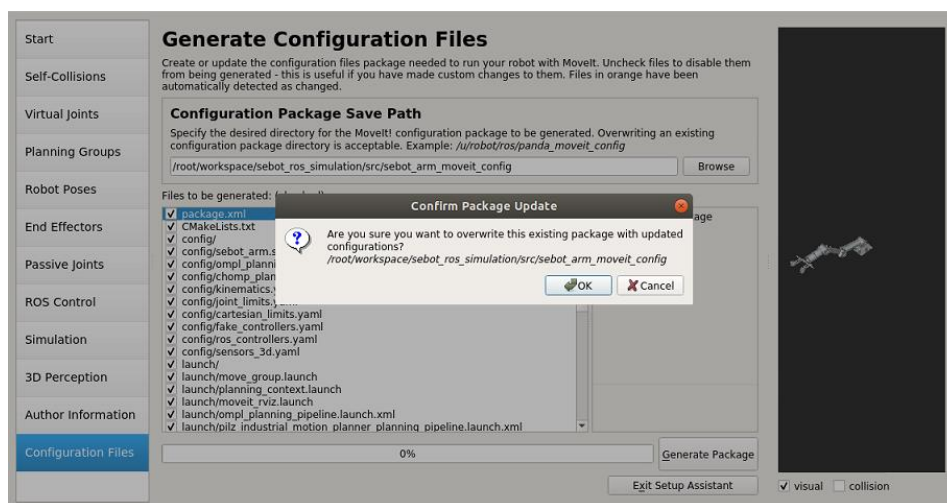


图 1-48 配置文件已有的提示弹窗

在配置文件夹中，会生成如下图所示的内容。其中“`config`”文件夹保存的是相关的机械臂配置，“`launch`”文件夹保存的是相关的启动文件。这些文件可以在后续的其他功能中直接调用。

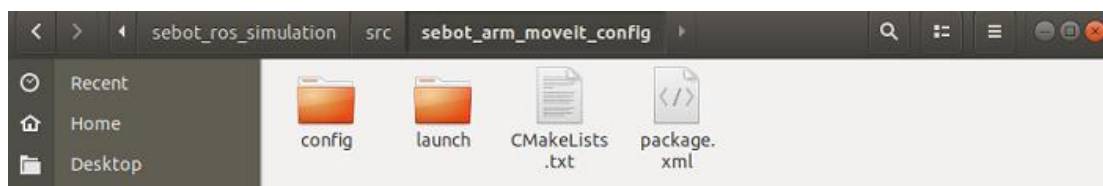


图 1-49 生成的配置文件

步骤 6：查看配置结果

机械臂配置完成后，可在终端中输入命令启动对应的机械臂模型。

打开一个新终端，进入工作空间，先执行 `source` 命令后再输入以下指令：

```
roslaunch sebot_arm_moveit_config demo.launch
```

```
root@paddlepi:~# cd workspace/sebot_ros_simulation/  
root@paddlepi:~/workspace/sebot_ros_simulation# source devel/setup.bash  
root@paddlepi:~/workspace/sebot_ros_simulation# roslaunch sebot_arm_moveit_config demo.launch
```

图 1-50 启动配置好的模型

程序运行后会打开 Rviz 软件，根据前面的步骤说明一样，在出现的界面中先把“Fixed Frame”改成“base_link”；然后添加“Robot Model”和“MotionPlanning”，接下来就可以观察到配置完成的机械臂模型了。

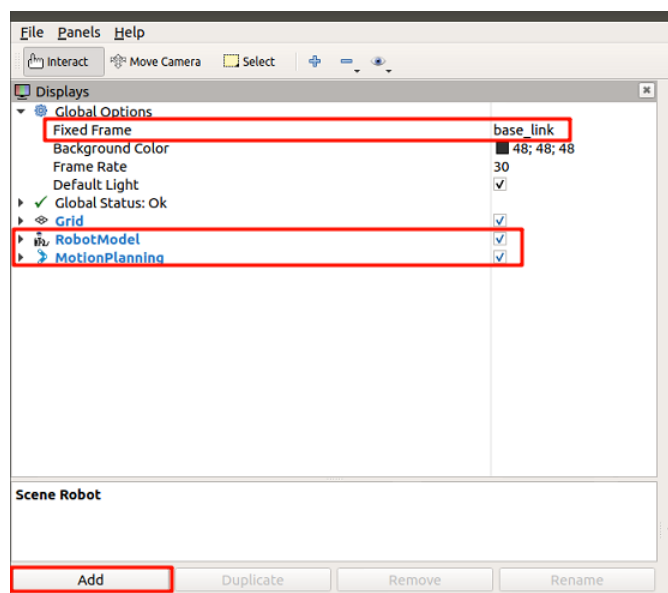


图 1-51 基本设置

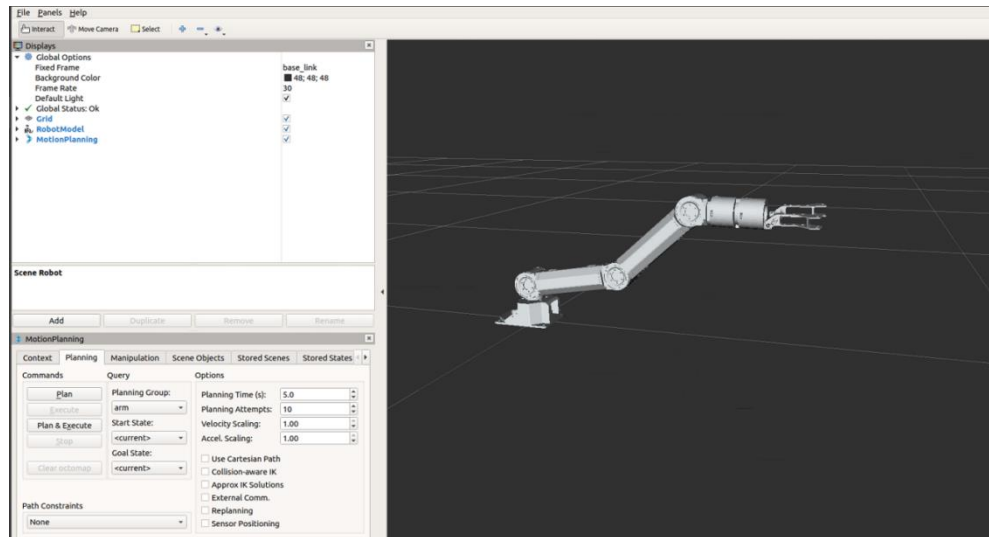


图 1-52 查看模型

配置完成后可在 MotionPlanning 中进行机械臂的控制，详细的内容将在下一章中说明。

5.拓展思考

机器人的 URDF 文件中可能涉及到的内容有哪些？

二、机械臂控制及路径规划仿真

1.实验描述

上次实训中我们完成了仿真环境以及机械臂自身的相关配置，通过 MoveIt 生成了对应的配置文件以及启动文件等。本次实训将详细说明仿真操作，即通过使用生成的相关文件，实现仿真环境中机械臂的指定控制。

机械臂的仿真操作主要包含基本控制与路径规划。基本控制指的是自由控制机械臂的各个关节，实现机械臂整体的姿态控制；路径规划指的是确定机械臂的起点和终点的位姿，然后由正逆运动学求解器在指定的关约束下通过路径规划算法计算出机械臂的运动路线。

2.实验目标

- (1) 找到仿真的启动文件，掌握 GUI 界面开启/关闭的方法，并控制机械臂的位姿；
- (2) 认识路径规划插件的基本使用方法，实现路径规划的基本操作，观察其实现效果。

3.实验步骤

实验开始前，先准备好个人 PC 及商服机器人，个人 PC 可通过远程连接方式访问商服机器人上的工程文件。

步骤 1：修改启动文件

通过 MoveIt 完成配置以后，最终会在指定的路径下生成一系列的相关文件。上次实训中，生成的文件所在路径为 `workspace\sebot_ros_simulation\src\sebot_arm_moveit_config`。

修改启动文件时，可在商服机器人本地直接打开，也可以在个人 PC 上通过远程连接方式打开。打开上述的文件夹后可在其中看到下图所示的文件内容。

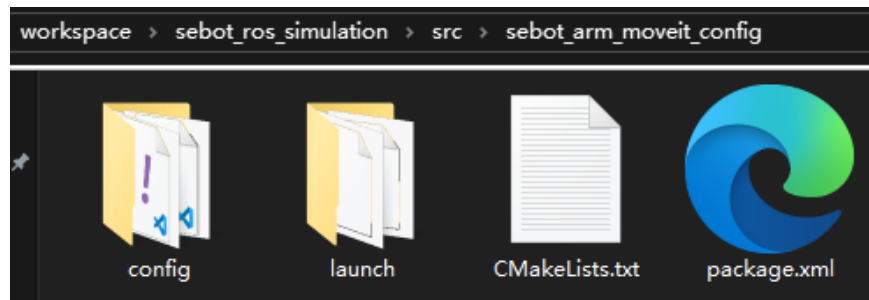


图 2-1 配置文件

config 文件夹中存放的是相关的配置内容，无需修改；launch 文件夹中存放的是启动文件，打开此文件夹，找到文件“demo.launch”并打开，此文件即启动文件。

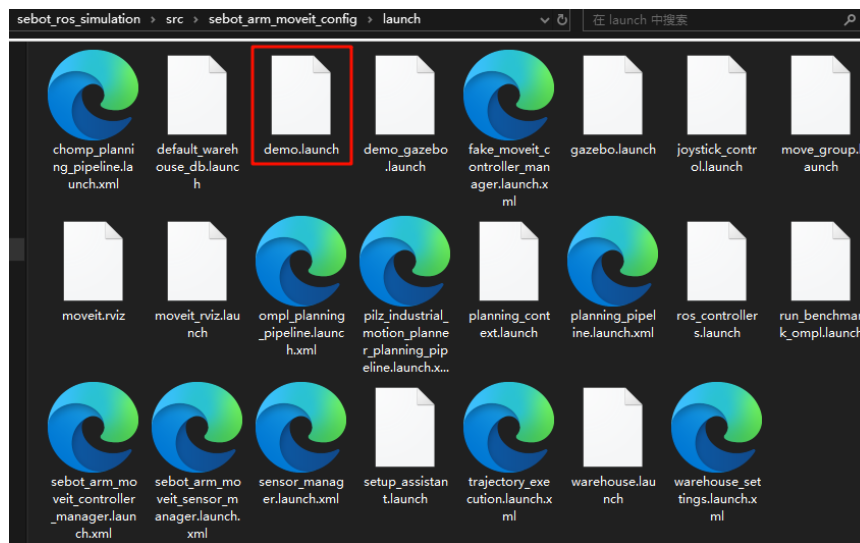


图 2-2 启动文件

此启动文件的作用主要有：

(1) 加载对应的配置内容。程序启动时往往需要从多个文件中读取对应的信息，通过启动文件读取的方式可以实现灵活的调用，从而不需要重新编写参数内容。

(2) 配置并启动多个节点。运行仿真功能时，不是仅有一个程序启动，而是多个程序协同工作。通过启动文件调用其他节点，可以方便程序之间的嵌套启动，从而实现功能的快速启动。

此文件中需要注意两行内容，如下图所示。

```

1 <launch>
2
3 <!-- specify the planning pipeline -->
4 <arg name="pipeline" default="ompl" />
5
6 <!-- By default, we do not start a database (it can be large) -->
7 <arg name="db" default="false" />
8 <!-- Allow user to specify database location -->
9 <arg name="db_path" default="$(find sebot_arm_moveit_config)/default_warehouse_mongo_db" />
10
11 <!-- By default, we are not in debug mode -->
12 <arg name="debug" default="false" />
13
14 <!-- By default, we will load or override the robot_description -->
15 <arg name="load_robot_description" default="true"/>
16
17 <!-- Set execution mode for fake execution controllers -->
18 <arg name="fake_execution_type" default="interpolate" />
19
20 <!--
21 By default, hide joint_state_publisher's GUI
22
23 MoveIt!'s "demo" mode replaces the real robot driver with the joint_state_publisher.
24 The latter one maintains and publishes the current joint configuration of the simulated robot.
25 It also provides a GUI to move the simulated robot around "manually".
26 This corresponds to moving around the real robot without the use of MoveIt.
27 -->
28 <arg name="use_gui" default="true" />
29 <arg name="use_rviz" default="true" />
30

```

图 2-3 启动文件的代码

这两行代码的作用是在启动程序时，选择是否开启 GUI 界面和 Rviz 界面，前者提供了机械臂控制的交互界面，后者则是打开 Rviz 并在其中观察机械臂的运动状态。使用时只需修改 default 的值为 true 或 false 即可控制是否开启 GUI 和 Rviz 界面。

此次实训时，需要确保这两项的值均为 true，这样在启动仿真程序时，就会自动打开 GUI 控制面板以及 Rviz 可视化软件。修改完成后记得保存文件。

```

<arg name="use_gui" default="true" />
<arg name="use_rviz" default="true" />

```

图 2-4 修改内容

步骤 2：启动仿真程序

本步骤可在商服机器人本地操作，也可在个人 PC 上通过远程连接机器人方式进行。

打开一个新终端，进入工作空间 sebot_ros_simulation 下，先执行 source 命令：

```
source devel/setup.bash
```

接着在终端输入以下指令：

```
roslaunch sebot_arm_moveit_config demo.launch
```

```

root@paddlepi:~# cd workspace/sebot_ros_simulation/
root@paddlepi:~/workspace/sebot_ros_simulation# source devel/setup.bash
root@paddlepi:~/workspace/sebot_ros_simulation# roslaunch sebot_arm_moveit_conf
g demo.launch

```

图 2-5 启动仿真

指令运行后会自动开启 Rviz 以及 GUI 控制界面。

Rviz 界面与上次实训中的一致。在 Rviz 窗口的左上方，还可以选择 **interact**（交互）模式和 **Move Camera**（移动摄影机）模式，前者可以对机械臂实现灵活的拖拽，后者只能调整观看的视角，此次实训中建议选择 **Move Camera**（移动摄影机）模式。

另外注意在 Rviz 中添加 **RobotModel** 和 **MotionPlanning** 两个插件。

RobotModel 用于显示机器人的三维模型，并且直观地查看机器人的结构、关节和链接等，这些功能基于运动学原理。

MotionPlanning 用于机器人的运动规划和仿真运行。其允许用户为机器人定义路径、设置目标点，并可视化观察机器人的运动轨迹。使用 **MotionPlanning** 插件时，可以配置相关参数并启动路径规划。启动后就可以在 RViz 界面中看到机器人的运动规划结果，包括从当前位置到目标位置的路径规划、避障策略以及预期的运动轨迹等。

操作模式选择及插件配置的结果如下图所示。

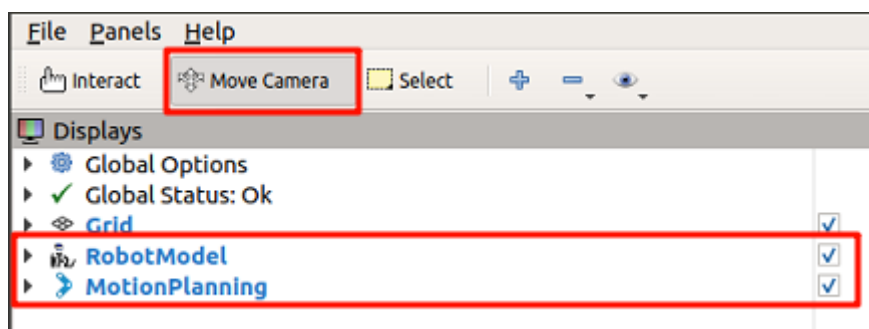


图 2-6 模式选择及插件添加

启动后的 GUI 控制窗口及界面如下图所示：

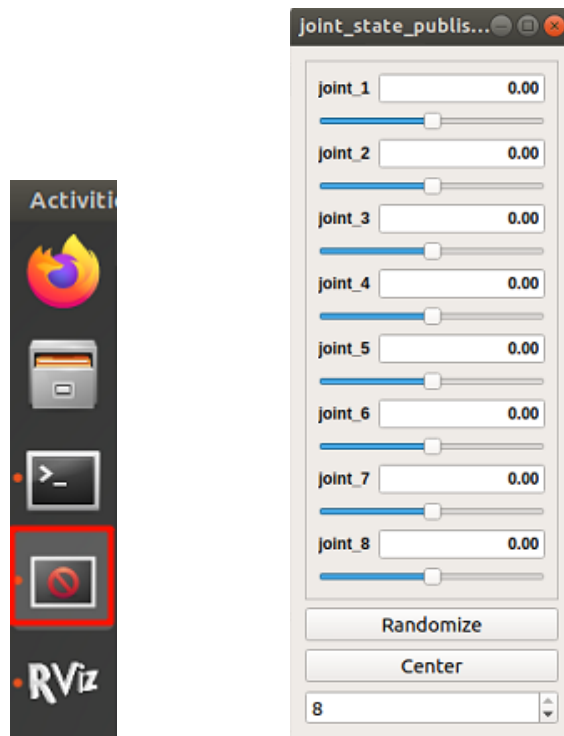


图 2-7 GUI 控制的窗口及界面

步骤 3：机械臂位姿控制

机械臂的位姿控制主要是通过调节各关节的参数，实现机械臂的位姿调整。前面的机械臂配置过程中已经设定好了关节间的运动学关系以及每个关节的约束，此步骤中将可视化的调整不同关节的参数并查看其实时效果。

在启动的 Rviz 界面中，先将 MotionPlanning 功能取消打勾，调节机械臂位姿时不需要使用此插件。

展开 RobotModel 插件，确保 Visual Enabled 选项处于勾选状态，此功能用于可视化显示机械臂模型；Collision Enabled 选项按需勾选，此功能用于碰撞检测显示，打开后会显示机械臂发生碰撞的部分，但是也会增加模型计算的负荷，碰撞的效果是通过红色部分呈现的。

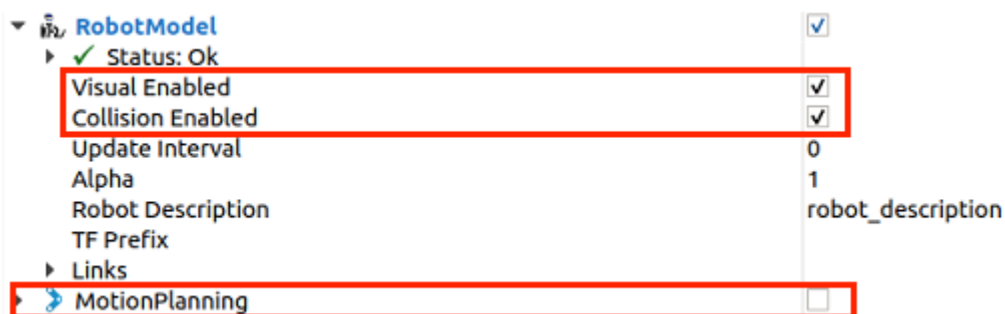


图 2-8 RobotModel 插件设置

接着在 Rviz 的界面中，将机械臂的位置调整到适宜的视角，然后在 GUI 界面中即可控制机械臂的位姿。

在 GUI 控制界面中，有 8 个滑杆用于操作机械臂的关节，通过调整滑杆可以修改关节旋转或滑动的数值，机械臂在可视化界面中也会实时产生对应的变化。

需要特别注意的是，joint 7 和 joint 8 是夹具的关节，这两个关节的控制类型为滑动，其他的关节均为转动，在调节这两个关节的数值时，不能太大，否则会偏离机械臂本体。其中 joint 7 的调节范围为-0.05~0.1，joint 8 的调节范围为-0.1~0.05，其他的关节可任意调整数值。

界面下方有两个按键：Randomize 表示随机角度，点击后每个关节的控制数值都会生成随机数，此步骤也需要注意 joint 7 和 joint 8 的数值，生成的随机数值可能会导致夹具脱离机械臂本体；Center 表示初始化数值，点击后机械臂可回归初始位姿。

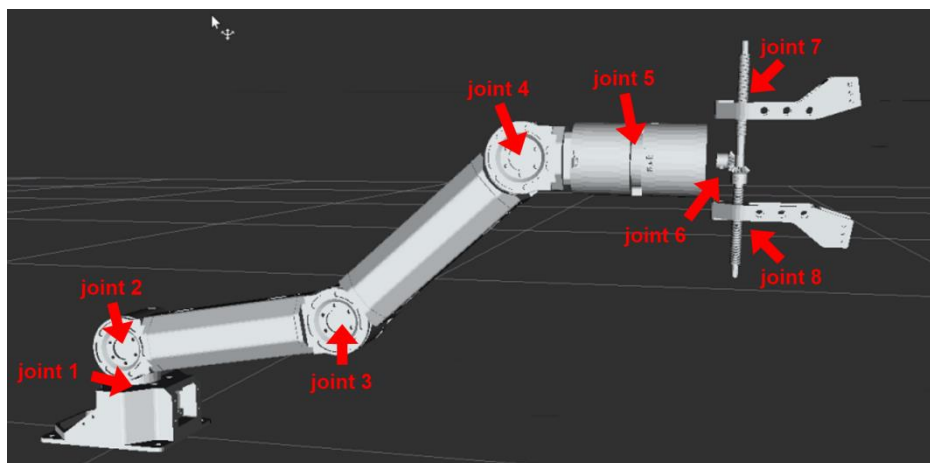


图 2-9 机械臂关节说明

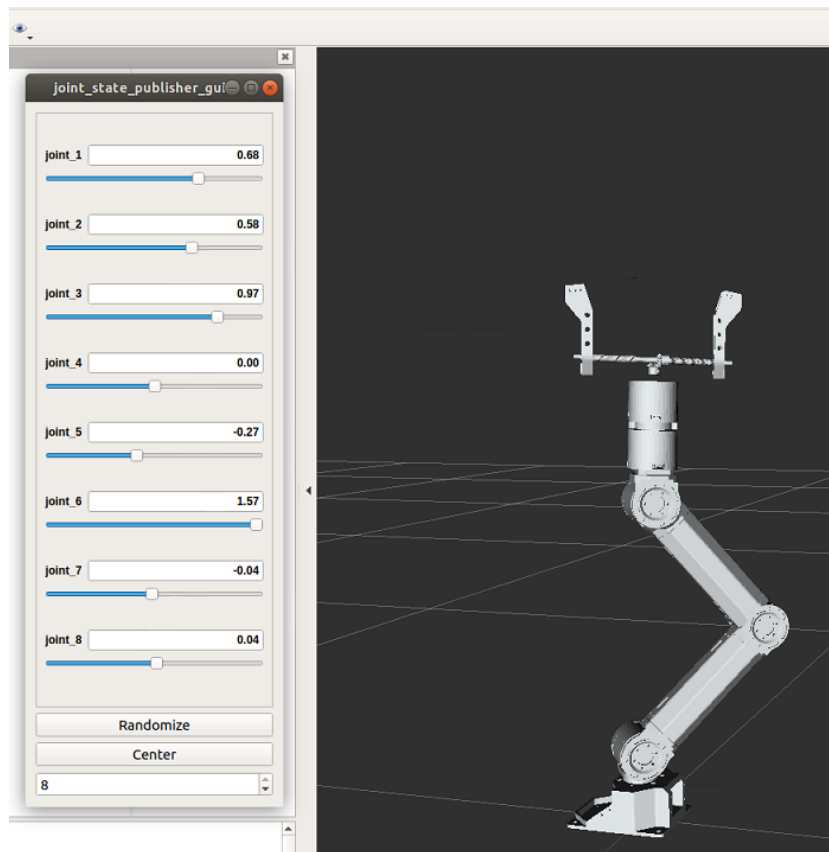


图 2-10 调整参数后的机械臂位姿

步骤 4：机械臂路径规划窗口选项

在进行路径规划时，同样也是在 Rviz 中进行操作。如果想要在启动程序时直接关闭前面的 GUI 控制界面，则可以在启动程序前，先将启动文件中 `use_gui` 的 `default` 值设为 `false`，保存修改之后再运行启动文件，即可只启动 Rviz 而不启动 GUI 控制界面了。

```
<arg name="use_gui" default="false" />
<arg name="use_rviz" default="true" />
```

图 2-11 关闭 GUI 界面

启动仿真程序的方法与步骤 2 一样，注意新开终端时要先在工作空间下执行 `source` 命令，已执行过此命令的终端无需再进行此操作。

在启动的 Rviz 界面中，先将 `RobotModel` 功能取消打勾，进行机械臂路径规划功能时不需要使用此插件。

展开 `MotionPlanning` 插件，下面有多个选项卡，以下介绍几个需要使用的选项。



图 2-12 MotionPlanning 插件

(1) Scene Robot

此选项卡用于控制机械臂模型，与前一步骤中的操作类似，勾选 Show Robot Visual 即可显示机械臂的真实模型，勾选 Show Robot Collision 即可显示机械臂的碰撞检测。在此次实训中，此两项功能可按需勾选。在后续的操作中，如果只想观察路径规划的结果，则可以关闭此模型，如果想要观察机械臂的真实运动，则需要开启此模型。

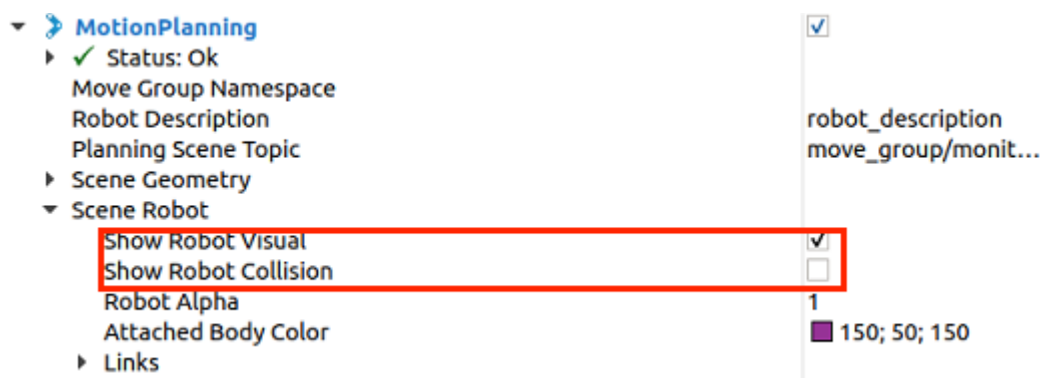


图 2-13 Scene Robot 选项卡

(2) Planning Request

此选项用于设置路径规划时的相关请求，Query Start State 用于显示机械臂路径规划时的起点位姿，Query Goal State 用于显示机械臂路径规划的终点位姿。在后续的操作中，起点位姿仅表示路径规划时的预设起点位姿，并不代表机械臂本体的真实位姿，而终点位姿既表示路径规划时的预设终点位姿，也表示机械臂本体最终将要运动到的终点位姿。因此，在进行路径规划和实际的机械臂运动时，两者的起点位姿可能不同，但终点位姿一定相同。



图 2-14 Planning Request 选项卡

如下图所示，绿色机械臂即为路径规划起点的机械臂位姿，橘色机械臂即为路径规划终点的机械臂位姿，如果起点位姿和终点位姿相同，则可能无法观察到两种颜色的机械臂。机械臂路径规划的过程即为从绿色形态转变为橘色形态，机械臂本体的运动起点可能不与绿色部分相同，终点一定与橘色部分相同。

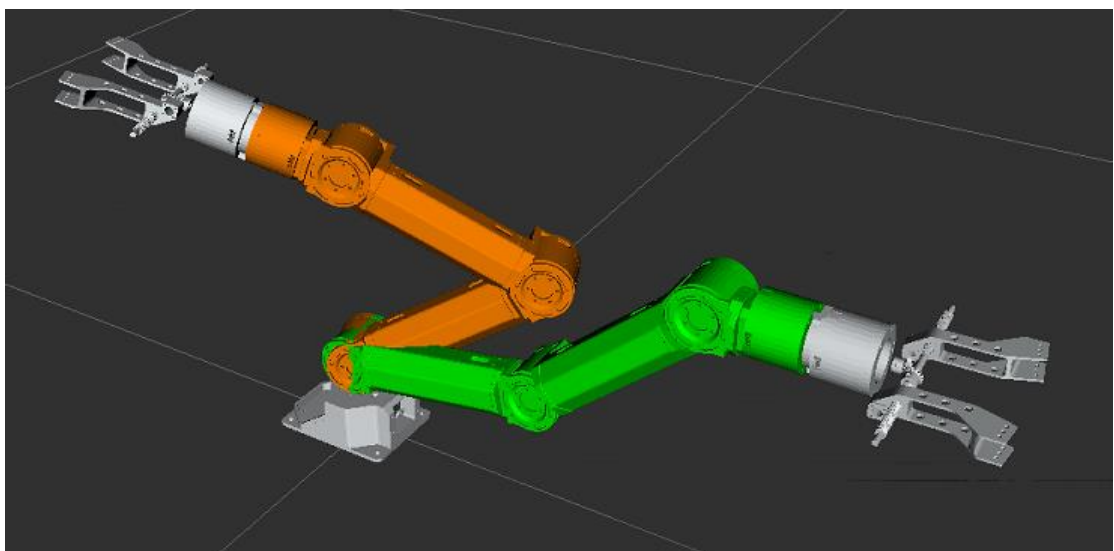


图 2-15 路径规划的起点和终点

(3) Planned Path

此选项用于设置路径规划过程中的可视化轨迹，勾选 Show Robot Visual 后，即可在路径规划的过程中观察到机械臂的实时位姿。如果未勾选此项，则无法观察机械臂的实时状态；勾选 Show Robot Collision 即可显示机械臂的碰撞检测，此功能可按需选择。



图 2-16 Planned Path 选项卡

下图中半透明灰色的机械臂即为机械臂路径规划过程中的实时位姿显示, 其运动过程便是从绿色的起点位姿运动到橘色的终点位姿。

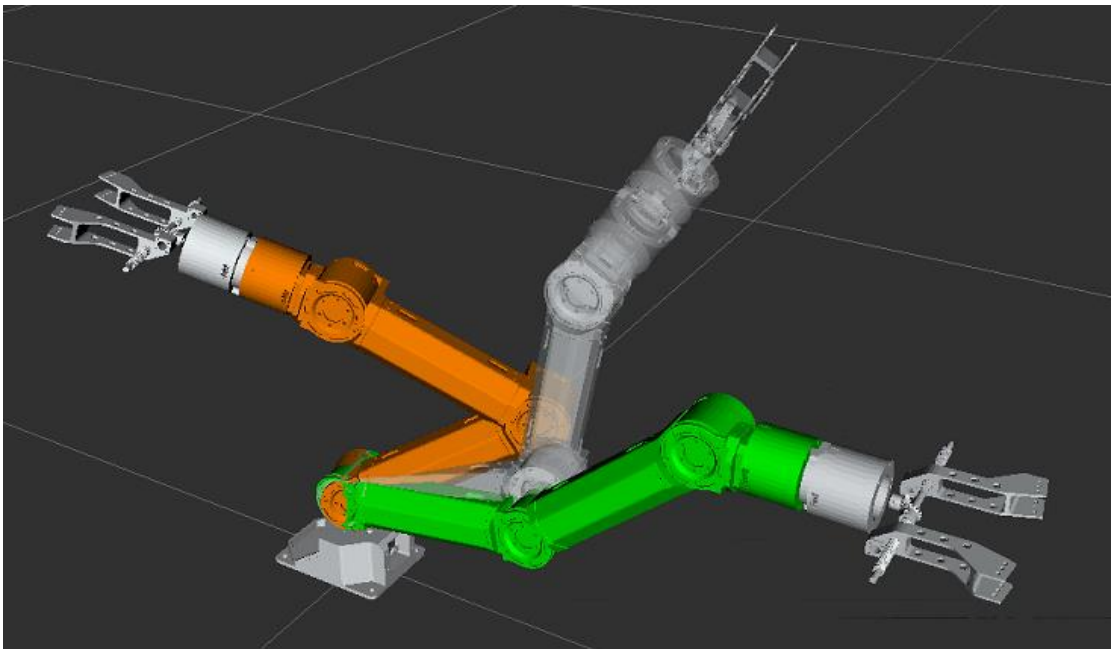


图 2-17 路径规划的轨迹显示

步骤 5: 机械臂路径规划操作

了解了基本选项的设置后, 接下来是进行路径规划的具体操作。

在 Rviz 软件左下角的 MotionPlanning 插件窗口中, 有多个选项卡, 我们需要使用的是其中的 Planning 和 Joints。

(1) Planning

此选项卡用于实现路径规划的具体操作。

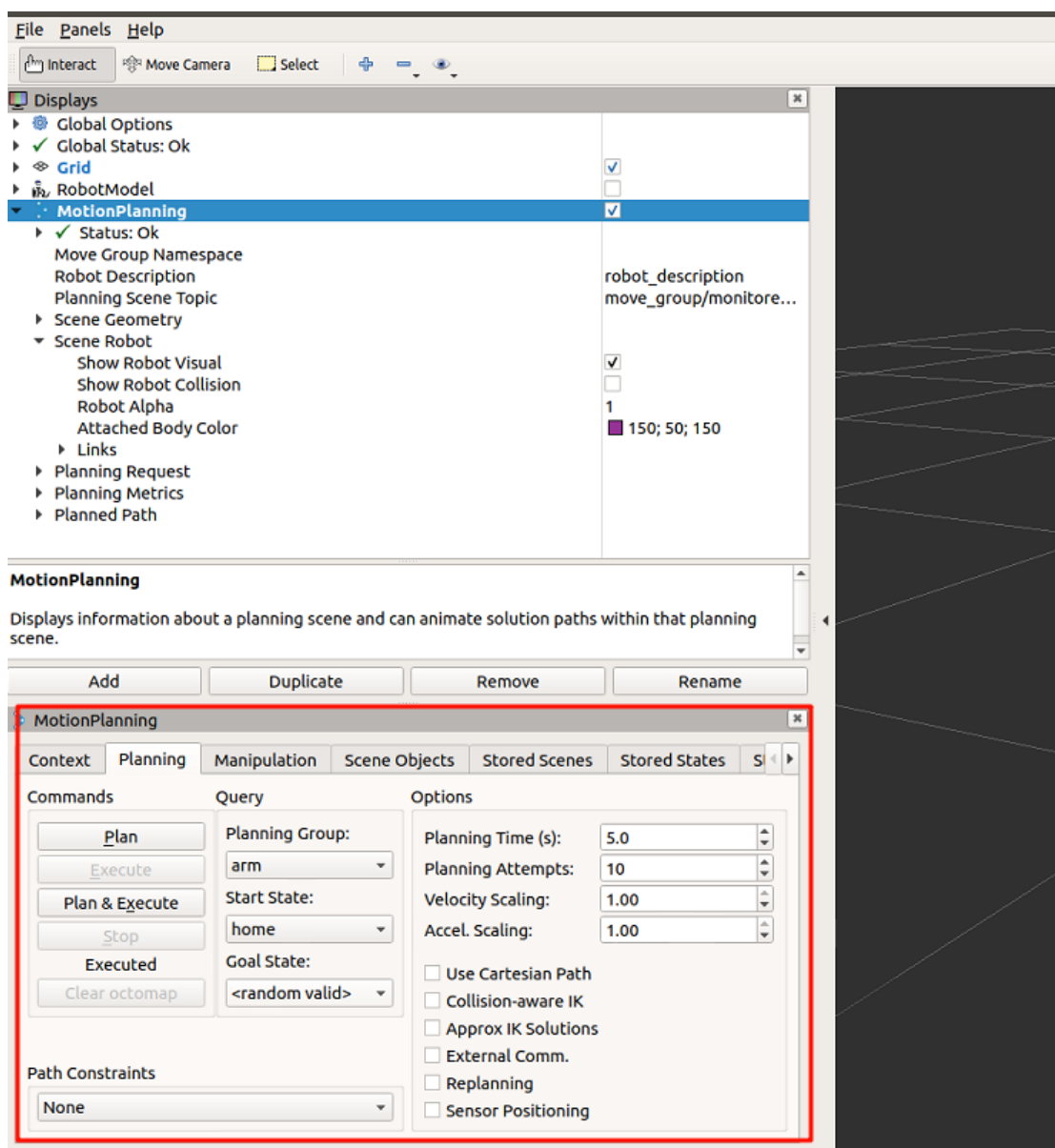


图 2-18 MotionPlanning 操作面板

以下按照路径规划功能的实现顺序来说明其中的主要选项卡：

① Query

如图 2-18，Query 选项卡用于设置路径规划的运动组以及起点和终点。

Planning Group 即执行动作的运动组，在前一次实训的机械臂配置中，分别有 arm 运动组和 gripper 运动组，前者是机械臂的躯干部分，后者是夹具部分，此次实训中选择 arm 即可。

Start State 即路径规划的起始状态，其中有五个预置选项以及提前布置好的配置动作。五个预置位姿为：random valid 表示随机且有效的位姿，即不发生碰撞的情况下随机生成的机械臂位姿；random 表示随机位姿，可能会发生结构间的碰撞；current 表示当

前位姿；same as goal 表示与终点位姿相同的位姿，previous 表示以前的位姿。预置选项下即为配置机械臂时设置好的位姿，例如图中的 home 即前面配置机械臂时设定好的动作。

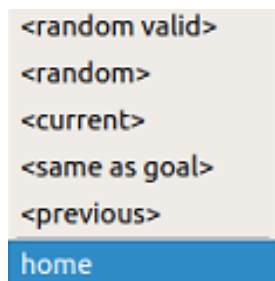


图 2-19 Start State 选项

Goal State 即路径规划的终点状态，其选项与 Start State 相同，仅将 same as goal 变成了 same as start，即与起点位姿相同的位姿。

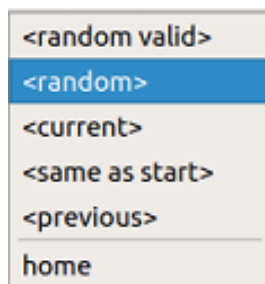


图 2-20 Goal State 选项

② Options

如图 2-18，Options 选项卡用于设置规划时的相关参数。

Planning Time 表示路径规划的时间；Planning Attempts 表示路径规划尝试次数；Velocity Scaling 表示速度设置，Accel Scaling 表示加速度设置。以上这些参数使用默认设置即可。

下面还有一个 Collision-aware IK 选项用于显示碰撞检测的结果，按需勾选即可。

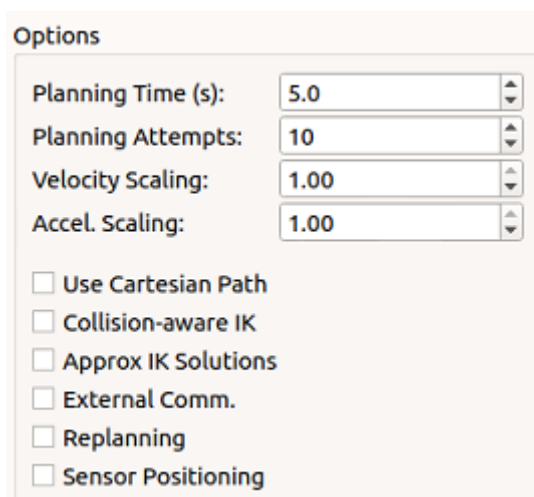


图 2-21 Options 操作面板

③ Commands

如图 2-18，Commands 选项卡用于直接控制路径规划，主要分为 Plan 和 Execute，前者表示进行路径规划的计算，执行时会按照预设的起点和终点进行规划功能，同时出现一个半透明的机械臂模型用于显示实时路径；后者表示执行功能，执行时控制的是机械臂的本体，同时从当前位置移动到终点位置。由于 Plan 和 Execute 按键控制的对象不同，因此执行的具体效果会出现差别。使用时按需选择，如果只想观察从起点位姿到终点位姿的实时运动，则点击 Plan 即可；如果要观察机械臂本体运动到终点位姿的实时运动，则点击 Execute 即可；二者均需要，则点击 Plan&Execute。实际使用时一般选择 Plan 和 Plan&Execute 即可，按键 Execute 可能受限于机械臂的状态无法使用。

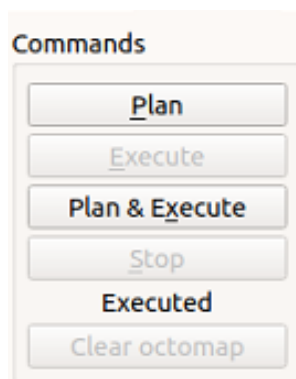


图 2-22 Commands 操作面板

如下图所示，绿色机械臂为路径规划起点位姿，橘色机械臂为路径规划终点位姿，半透明灰色机械臂为路径规划的实时位姿，灰色机械臂为机械臂本体的实时位姿，路径规划的实时位姿与本体的实时位姿可能不一致。

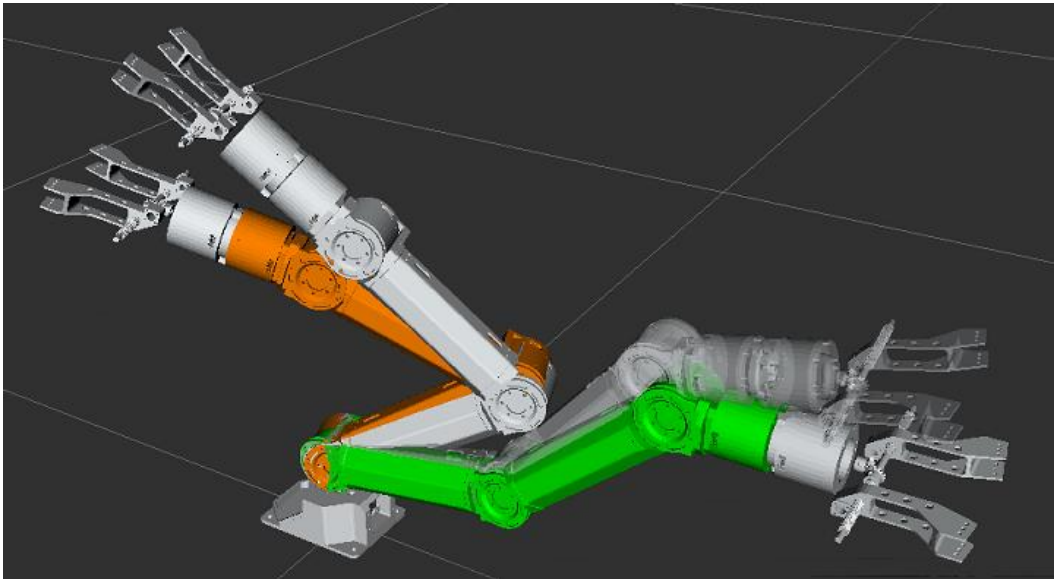


图 2-23 机械臂路径规划执行过程

(2) Joints

在 Rviz 中, Joints 选项卡用于控制终点位姿的调节, 不过仅限于当前所选的动作组, 操作的方法也是通过滑杆调节, 调节数值后窗口中终点(橘色机械臂)的位姿就会发生对应的改变。

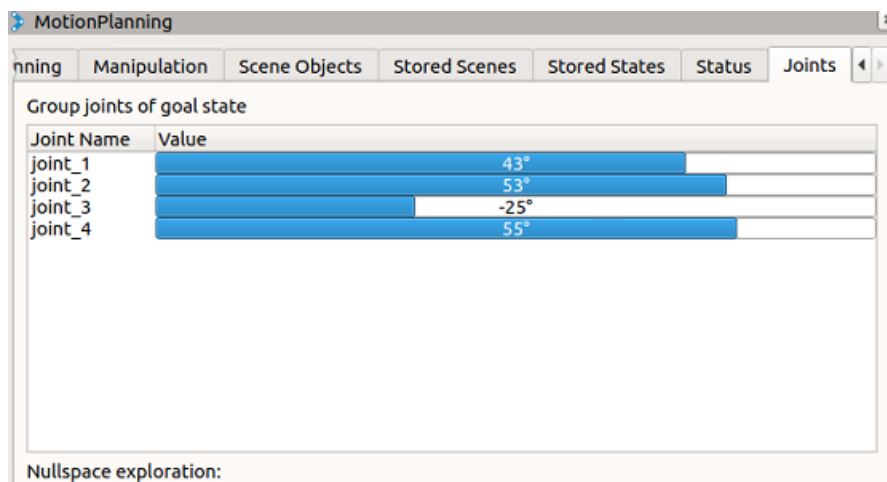


图 2-24 Joints 操作面板

(3) 操作顺序

进行路径规划操作时, 先选择运动组以及起点和终点的位姿, 终点位姿可以直接选择, 也可以在 Joints 选项卡里手动调整; 然后设置相关参数, 建议使用默认参数, 碰撞检测可按需开启/关闭; 最后进行规划和执行的操作, Plan 为规划, Execute 为执行, Plan&Execute 为规划并执行。单击后即可在窗口中看到对应的机械臂运动。执行 Plan 功能时, 机械臂可能会重复进行路径规划, 此时单击一下 Plan&Execute 按键即可。

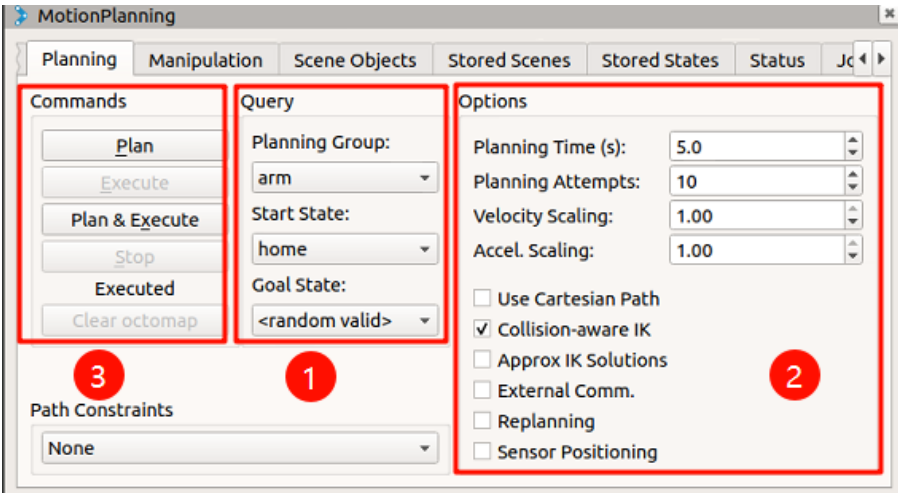


图 2-25 路径规划操作顺序

Group joints of start state

Joint Name	Value
joint_1	23°
joint_2	20°
joint_3	89°
joint_4	51°

图 2-26 手动调整终点位姿

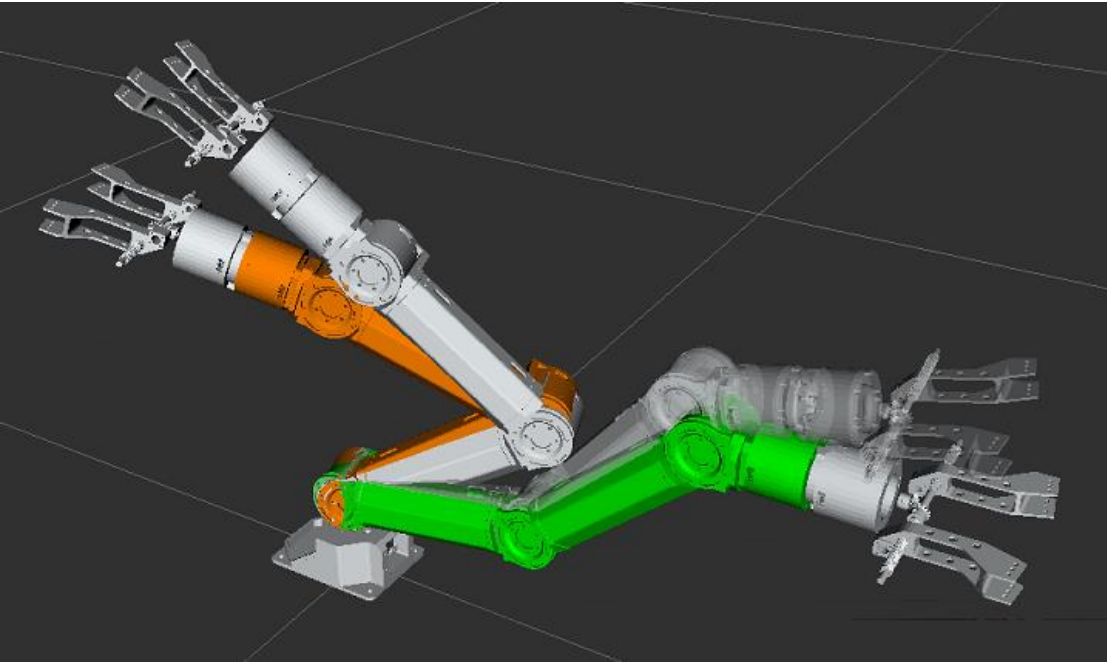


图 2-27 机械臂执行 Plan&Execute 的效果

操作时需要注意每次规划时要把起点和终点设置好，并且每次运行 Execute 功能后，机械臂本体的位姿都会发生改变。

4.拓展思考

在 RobotModel 和 MotionPlanning 插件的其他选项卡中，还可能具有哪些功能呢？

三、编程控制仿真机械臂

1.实验描述

上次实训中，我们通过对应的软件操作，实现了机械臂的运动控制以及路径规划的仿真。除了学习软件的基本操作外，更为关键的是理解了机械臂路径规划的控制逻辑，即选择运动组、设置起点和终点的位姿、设置过程中的相关参数、执行仿真并观察结果，调整参数再优化。

本次实训中，我们将通过编写程序代码的形式，实现机械臂路径规划仿真的功能。通过代码编写的方式进行实操，一方面可以更好的理解其逻辑思路，另一方面还可以理解 ROS 系统的一些相关控制。本次操作除了基本功能外，还可以在代码中实现拓展操作，实现比软件操作更为复杂的效果。

2.实验目标

- (1) 理解通过程序控制仿真机械臂的路径规划的代码逻辑；
- (2) 掌握机械臂路径规划仿真的程序运行操作方法；
- (3) 练习机械臂路径规划的仿真功能，并拓展优化源码，实现更优效果。

3.实验步骤

本次实训中由于相关文件较多，因此使用已配置完成的工程文件夹，具体路径为 `workspace\sebot-t710-competition`，并且在终端的操作中，也无需像前面的步骤一样，每次开启新终端都要先运行 `source` 命令。需要注意的是，如果在同一个终端中，已经进行了前面所学习的 `source` 操作，那么在运行本次实训的操作时，需要重新开启终端，否则会影响使用。

步骤 1：理解路径规划仿真的代码逻辑

通过在商服机器人本地或本地 PC 远程共享商服机器人文件的方式打开此次实训使用的代码文件，具体路径为 `workspace\sebot-t710-competition\sebot_ros_kits\src\sebot_driver\sebot_talon\scripts`。进入此路径后，打开代码文件“`talon_simulation.py`”

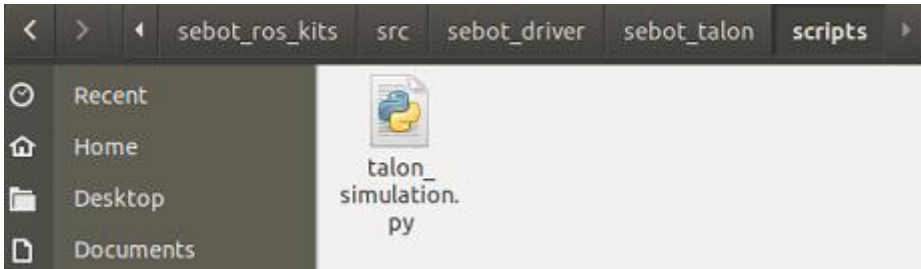


图 3-1 文件路径

如下图为打开后的代码内容。

```
talon_simulation.py 3
root > root > workspace > sebot-tf10-competition > sebot_ros_kits > src > sebot_driver > sebot_talon > scripts > talon_simulation.py > ...
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3
4  """
5  @file      :talon_simulation.py
6  @Description :通过坐标位置来机械臂
7  @Date      :2024/03/28 19:29:59
8  @Autor     :HC
9  @Version   :v1.0
10 """
11
12 import rospy, sys
13 import moveit_commander
14 from geometry_msgs.msg import PoseStamped
15
16 class MoveItArm:
17     """
18     moveit机械臂控制
19     """
20     def __init__(self):
21
22         moveit_commander.roscpp_initialize(sys.argv) # 初始化moveit_commander, 并启动moveit节点
23         rospy.init_node('sebot_talon') # 节点名称
24
25         self.arm = moveit_commander.MoveGroupCommander('arm') # 机械臂主体部分
26         # self.gripper = moveit_commander.MoveGroupCommander('gripper') # 机械爪部分
27
28         self.reference_frame = 'base_link' # 设置目标位置所使用的参考坐标系
29
30         self.arm.set_pose_reference_frame(self.reference_frame) # 定位坐标系
31
```

图 3-2 代码内容

下面详细说明其程序逻辑：

(1) 导入相关库

程序运行前，需要先导入相关库以供后续调用，调用的库及作用如下：

rospy: rospy 是 ROS 的纯 Python 客户端库，它提供了 Python 程序员需要的接口，以便能够快速地与 ROS 主题（topic）、服务（service）和参数（param）进行交互。

sys: sys 库是 Python 标准库中的一个重要库，它提供了许多与 Python 解释器和运行环境相关的功能。

moveit_commander: moveit_commander 是 ROS 中 MoveIt 框架的一部分，它为 Python 提供了方便的接口，允许用户通过编程方式控制和规划机器人的运动。

geometry_msgs.msg: geometry_msgs.msg 是 ROS 中的一个消息定义包，它包含了一系列与几何形状和运动学相关的消息类型。PoseStamped 是其中的一个消息类型，它主要用于表示机器人相对于全局坐标系的位置和姿态。

```
import rospy, sys
import moveit_commander
from geometry_msgs.msg import PoseStamped
```

图 3-3 导入库

(2) 创建机械臂控制类 MoveItArm

此类的作用是为了方便后续在主程序中的调用，其中又包含了两个函数，分别是 init () 和 coordCtrl(target_pose)。

```
class MoveItArm:
    """
    moveit机械臂控制
    """
```

图 3-4 定义类

① init () 函数

此函数的作用是当实例化 MoveItArm 类时，执行其中的功能。

函数运行时，首先是启动对应的节点并进行初始化操作，以便后续调用；接着是定义机械臂进行路径规划时需要的全部参数，包括运动组、坐标系以及功能设置等，最后是控制机械臂从当前位置回到初始化位置。需要注意的是，初始化位置是在配置 MoveIt 时定义好的位置，因此调用函数时的参数为“home”，如果定义了其他位置，也可以修改此参数。

详细的代码内容及重要参数如下图所示。

```
def __init__(self):
    moveit_commander.roscpp_initialize(sys.argv)    # 初始化moveit_commander, 并启动moveit节点
    rospy.init_node('sebot_talon')                # 节点名称

    self.arm = moveit_commander.MoveGroupCommander('arm')    # 机械臂主体部分
    # self.gripper = moveit_commander.MoveGroupCommander('gripper') # 机械爪部分

    self.reference_frame = 'base_link'              # 设置目标位置所使用的参考坐标系

    self.arm.set_pose_reference_frame(self.reference_frame) # 定位坐标系

    self.arm.allow_replanning(True)                 # 是否允许重复规划路径(最多五次)

    self.arm.set_goal_joint_tolerance(0.3)          # 设置位置(单位: 米)允许误差

    self.arm.set_start_state_to_current_state()     # 设置机器臂当前的状态作为运动初始状态

    self.arm.set_named_target('home')               # 控制机械臂先回到初始化位置
    self.arm.go()
    rospy.sleep(1)
```


图 3-5 inti () 函数

② coordCtrl(target_pose)函数

此函数的作用是根据输入的位姿信息，设定末端需要到达的目标点的位置以及自身的姿态，然后通过路径规划器的计算得到具体的运动路线，最终机械臂将按照此路线进行运动。运动完成后机械臂会暂停 2 秒。

```
def coordCtrl(self,target_pose):
    """
    通过坐标控制机械臂
    param: target_pose 目标位姿
    """
    self.arm.set_pose_target(target_pose, self.arm.get_end_effector_link()) # 获取机械臂末端运动目标位姿

    traj = self.arm.plan() # 规划运动路径
    self.arm.execute(traj) # 执行运动路径
    rospy.sleep(2) # 等待运动完成
```

图 3-6 coordCtrl () 函数

(3) 主函数

此函数的作用是实现程序的运行。函数中先实例化 MoveItArm 类，接着是设置目标点的位姿信息，此信息是以 PoseStamped 类型设置的。在此消息中，header.frame_id 用于指定参考的坐标系，header.stamp 用于表示时间戳，pose.position 表示参考坐标系下的空间坐标，pose.orientation 表示参考坐标系下的姿态信息（即空间方向），其表现形式为四元数。

目标点位姿信息设置完成后则调用 coordCtrl () 函数实现机械臂的路径规划。路径规划执行完成后，函数会退出 MoveIt 并释放系统资源。

```
if __name__ == "__main__":
    moveItArm = MoveItArm()
    # 设置机械臂工作空间中的目标位姿，位置使用x、y、z坐标描述，
    # 设置机械臂工作空间中的目标位姿
    target_pose = PoseStamped()
    target_pose.header.frame_id = moveItArm.reference_frame
    target_pose.header.stamp = rospy.Time.now()
    target_pose.pose.position.x = -0.0996125
    target_pose.pose.position.y = -0.0204184
    target_pose.pose.position.z = 0.109637
    target_pose.pose.orientation.x = -0.297784
    target_pose.pose.orientation.y = -0.642886
    target_pose.pose.orientation.z = 0.643069
    target_pose.pose.orientation.w = -0.290663

    moveItArm.coordCtrl(target_pose) # 通过坐标控制机械臂

    # 关闭并退出moveit
    moveit_commander.roscpp_shutdown()
    moveit_commander.os._exit(0)
```

图 3-7 主函数

代码功能理解后即可进入到实际的操作环节，在最后的步骤中还会对此代码文件进

行修改和运行。

步骤 2：运行 Rviz 与 GUI

Rviz 和 GUI 启动文件的路径为：workspace\sebot-t710-competition\sebot_ros_kits\src\sebot_simulator\launch\sebot_talon.launch。打开此文件即可看到具体内容，其内容与上一次实训一致。

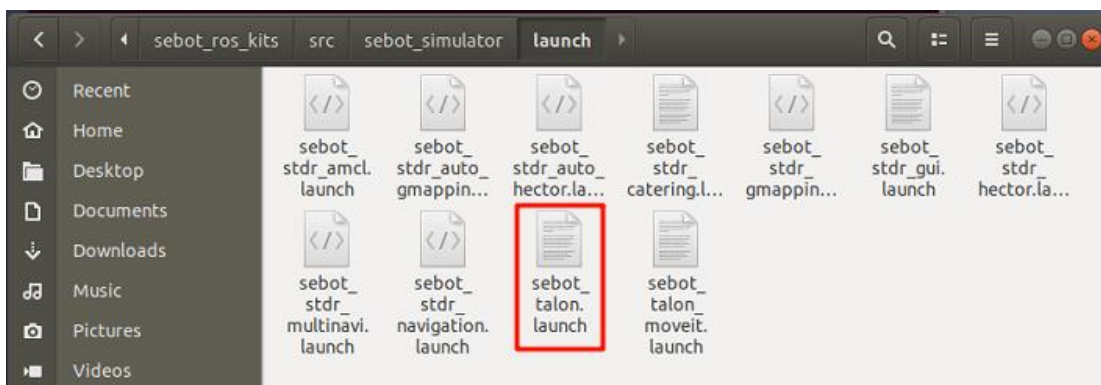


图 3-8 Rviz 和 GUI 的启动文件

本次实训中，需要将 Rviz 和 GUI 的功能都开启，启动前注意启动文件中的参数设置。

```
root > root > workspace > sebot-t710-competition > sebot_ros_kits > src > sebot_simulator > launch > sebot_talon.launch
1 <launch>
2
3 <!-- specify the planning pipeline -->
4 <arg name="pipeline" default="ompl" />
5
6 <!-- By default, we do not start a database (it can be large) -->
7 <arg name="db" default="false" />
8 <!-- Allow user to specify database location -->
9 <arg name="db_path" default="$(find sebot_talon_moveit)/default_warehouse_mongo_db" />
10
11 <!-- By default, we are not in debug mode -->
12 <arg name="debug" default="false" />
13
14 <!-- By default, we will load or override the robot_description -->
15 <arg name="load_robot_description" default="true"/>
16
17 <!-- Set execution mode for fake execution controllers -->
18 <arg name="fake_execution_type" default="interpolate" />
19
20 <!--
21 By default, hide joint_state_publisher's GUI
22
23 MoveIt!'s "demo" mode replaces the real robot driver with the joint_state_publisher.
24 The latter one maintains and publishes the current joint configuration of the simulated robot.
25 It also provides a GUI to move the simulated robot around "manually".
26 This corresponds to moving around the real robot without the use of MoveIt.
27 -->
28 <arg name="use_gui" default="true" />
29 <arg name="use_rviz" default="true" />
```

图 3-9 Rviz 和 GUI 的启动文件内容

打开新终端，进入到工作空间 workspace\sebot-t710-competition 路径下，输入以下命令运行 Rviz 和实时控制的 GUI。

```
roslaunch sebot_simulator sebot_talon.launch
```

```
root@paddlepi:~# cd workspace/sebot-t710-competition/
root@paddlepi:~/workspace/sebot-t710-competition# roslaunch sebot_simulator sebot_t_talon.launch
```

图 3-10 终端输入 Rviz 和 GUI 的启动命令

命令执行后会自动打开 Rviz 和 GUI，操作的方法与上次实训一致。Rviz 窗口左侧有折叠按钮，单击可以显示/折叠相关的插件及其信息。控制右侧的 GUI 可使 Rviz 中的机械臂产生对应的动作。

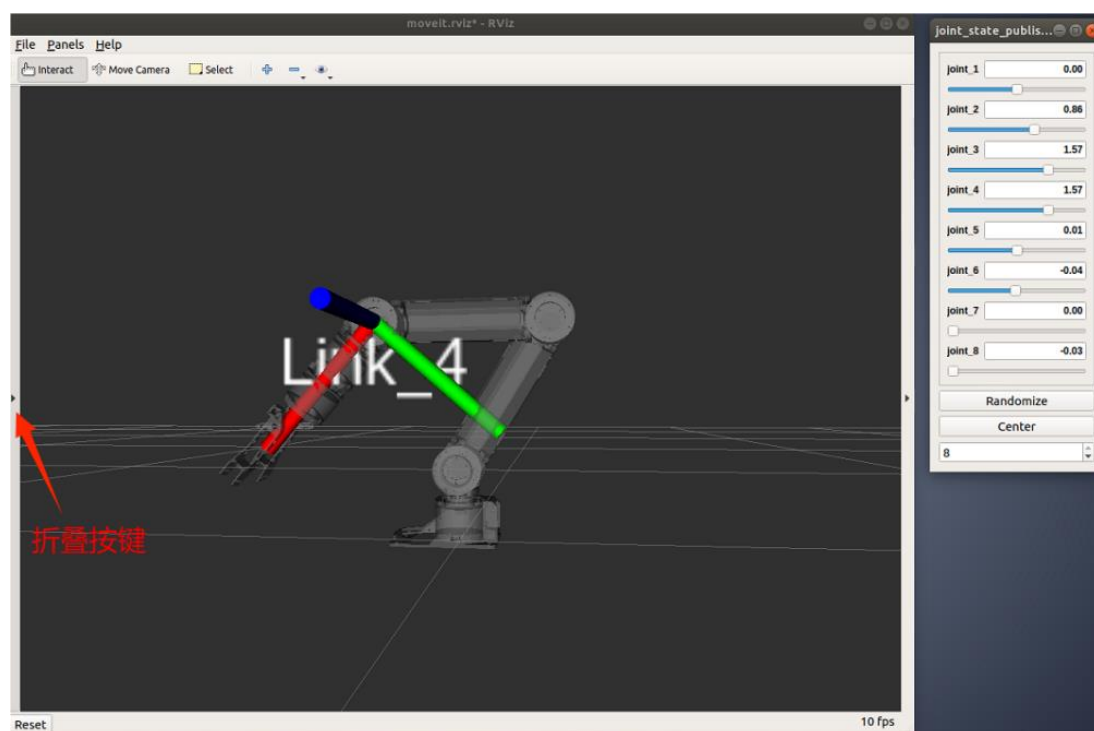


图 3-11 Rviz 和 GUI 启动后的界面

步骤 3：查看机械臂位姿参数

单击 Rviz 界面左侧的折叠按钮，可以查看当前的状态信息，选择插件中的“TF”，此插件用于 TF 坐标的相关功能，通过此功能可以查看机械臂每个关节在空间中的位姿参数。

在左侧的控制面板中，选中“TF”插件，展开“Frame”选项卡，选择末端关节 Link_4，展开 Link_4 下的“Position”和“Orientation”选项，前者为末端关节 Link_4 在固定参考坐标系中的空间坐标，后者为用四元数表示的末端关节 Link_4 在空间中的姿态。

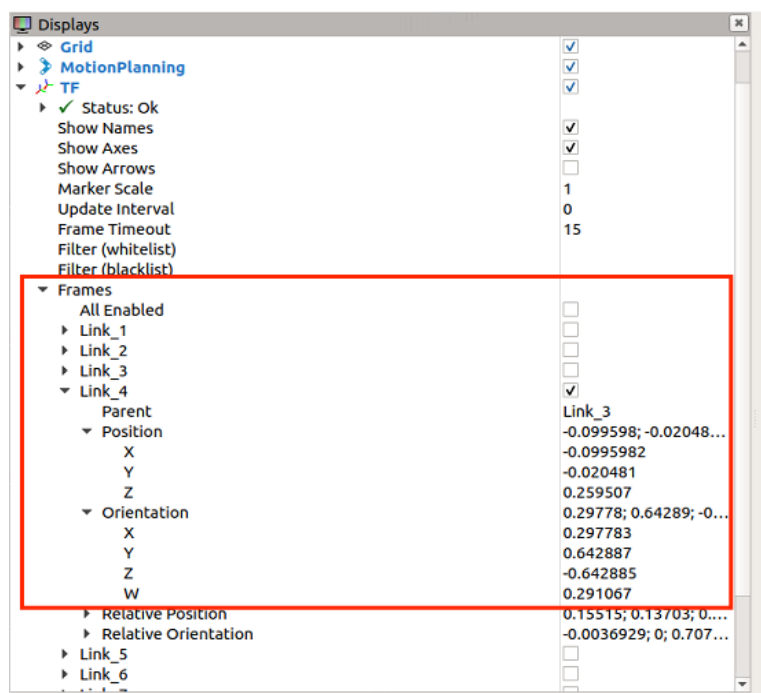


图 3-12 Rviz 中的关节位姿信息

除了在 Rviz 中查看机械臂关节的位姿信息外，在终端中也可以查看这些信息。

打开新终端，进入工作空间 `workspace\sebot-t710-competition` 路径后输入以下指令。

此命令中 `base_link` 表示参考坐标系，`Link_4` 表示需要显示位姿信息的关节，如果想要查看其他关节在参考坐标系下的位姿信息，可将 `Link_4` 替换成其他的关节名。

```
rosrun tf tf_echo base_link Link_4
```

命令运行后，终端中会持续输出 `Link_4` 的位姿信息并刷新。如下图所示，`time` 表示运行时间，`Translation` 表示变换后对应的空间坐标，`Quaternion` 表示姿态的四元数数据，`radian` 表示关节相对于参考坐标系的弧度数据，`degree` 表示关节相对于参考坐标系的角度数据。

```
root@paddlepi:~# cd workspace/sebot-t710-competition/
root@paddlepi:~/workspace/sebot-t710-competition# rosrn tf tf_echo base_link Link_4
At time 1712124698.878
- Translation: [-0.100, -0.020, 0.260]
- Rotation: in Quaternion [0.298, 0.643, -0.643, 0.291]
           in RPY (radian) [-1.577, 0.859, 3.128]
           in RPY (degree) [-90.347, 49.212, 179.242]
At time 1712124699.578
- Translation: [-0.100, -0.020, 0.260]
- Rotation: in Quaternion [0.298, 0.643, -0.643, 0.291]
           in RPY (radian) [-1.577, 0.859, 3.128]
           in RPY (degree) [-90.347, 49.212, 179.242]
At time 1712124700.579
- Translation: [-0.100, -0.020, 0.260]
- Rotation: in Quaternion [0.298, 0.643, -0.643, 0.291]
           in RPY (radian) [-1.577, 0.859, 3.128]
           in RPY (degree) [-90.347, 49.212, 179.242]
```

图 3-13 终端输出的关节位姿信息

需要停止终端输出数据时可以使用快捷键“Ctrl”+“C”，再次查看时则重新输入上述的输出位姿命令。

如果在查看位姿参数的过程中机械臂发生了运动，那么 Rviz 和终端中也会发生对应的数据变化。需要注意的是，终端中输出的信息与 Rviz 中的信息实际上是相同的，但是终端中输出的数据精度仅精确到 0.001，而 Rviz 中的数据精度则精确到了 0.000001。

步骤 4：运行仿真机械臂的控制命令

控制仿真机械臂进行路径规划实际上就是通过启动文件运行步骤 1 中的 python 代码文件。启动文件的路径为：workspace\sebot-t710-competition\sebot_ros_kits\src\sebot_simulator\launch\sebot_talon_moveit.launch。打开此文件即可看到对代码文件的调用。

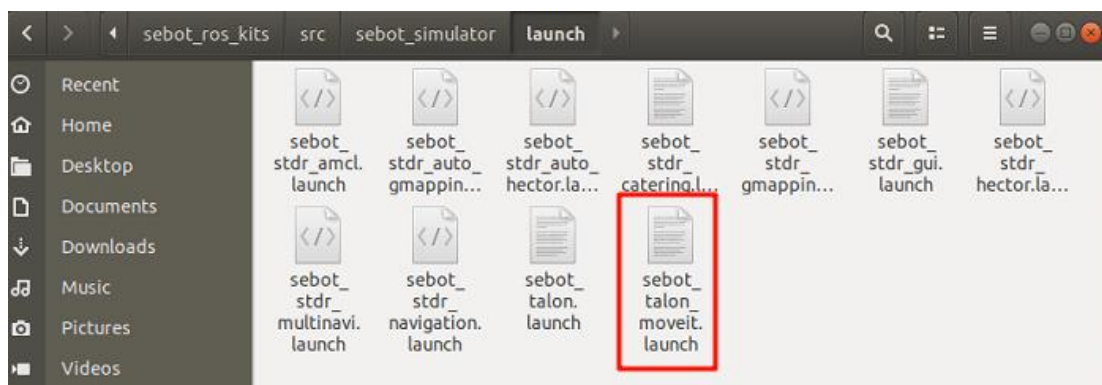


图 3-14 机械臂控制的启动文件路径

```
root > root > workspace > sebot-t710-competition > sebot_ros_kits > src > sebot_simulator > launch > sebot_talon_moveit.launch
1 <launch>
2 <!-- 启动机械臂控制节点 -->
3 <node pkg="sebot_talon" type="talon_simulation.py" name="talon_simulation"/>
4 </launch>
```

图 3-15 机械臂控制的启动文件内容

运行程序前，如果代码文件有修改，注意保存后再运行。

打开新终端，进入工作空间 workspace\sebot-t710-competition 路径后输入以下命令即可运行机械臂路径规划仿真的程序。

```
roslaunch sebot_simulator sebot_talon_moveit.launch
```

```
root@paddlepi:~# cd workspace/sebot-t710-competition/
root@paddlepi:~/workspace/sebot-t710-competition# roslaunch sebot_simulator sebot_talon_moveit.launch
```

图 3-16 终端输入机械臂控制指令

命令执行后, Rviz 中的机械臂会按照代码内容, 首先从当前位姿运动到初始位姿, 然后运动到代码指定的位姿。

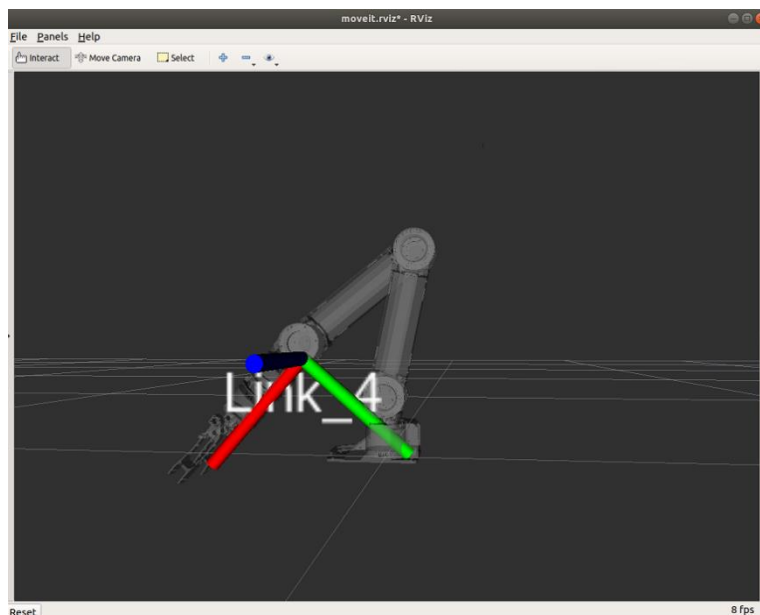


图 3-17 机械臂路径规划效果

步骤 5: 机械臂路径规划的代码优化拓展练习

前面步骤的进行均是基于已配置好的程序内容, 下面说明自定义路径的具体操作以及多点路径规划的提示。

操作前, 先将路径规划的代码文件 “talon_simulation.py” 复制一份作为备份, 防止源文件改动过多无法复原。备份完成后即可在源代码文件 “talon_simulation.py” 上进行修改和测试, 如果无法复原源文件, 则可以删除该文件并将备份文件修改成源文件的名称。

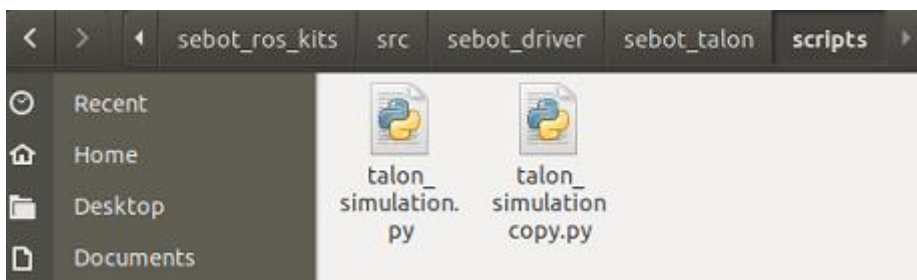


图 3-18 文件备份

(1) 自定义路径规划

根据机械臂运动学的相关知识, 本次路径规划的实现原理是基于运动学的逆解算, 即根据终点位姿计算出每个关节的运动参数, 然后控制机械臂的各个关节运动。但是需要注意的是, 机械臂并不一定能够实现空间中的任意一个位姿, 这是由于机械臂结构的约束导

致的。因此如果给出任意的目标位姿，机械臂不一定能够计算出对应的答案。想要实现正确的路径规划，就必须确保给定的目标位姿是机械臂可以实现的。

基于此原理，在进行路径规划的练习时，首先需要通过 GUI 界面调整机械臂到指定的位姿，然后将其数据填写到代码文件中并保存，最后在终端中运行控制指令后，即可观察到机械臂的路径规划。

以下是具体操作步骤：

① 启动 Rviz 和 GUI 控制后，通过 GUI 控制机械臂运动到指定的位姿，控制过程中可以在 Rviz 中看到机械臂的姿态以及其参数的变化。通过调整 GUI 确定好机械臂的目标位姿后，即可进入到下一步。

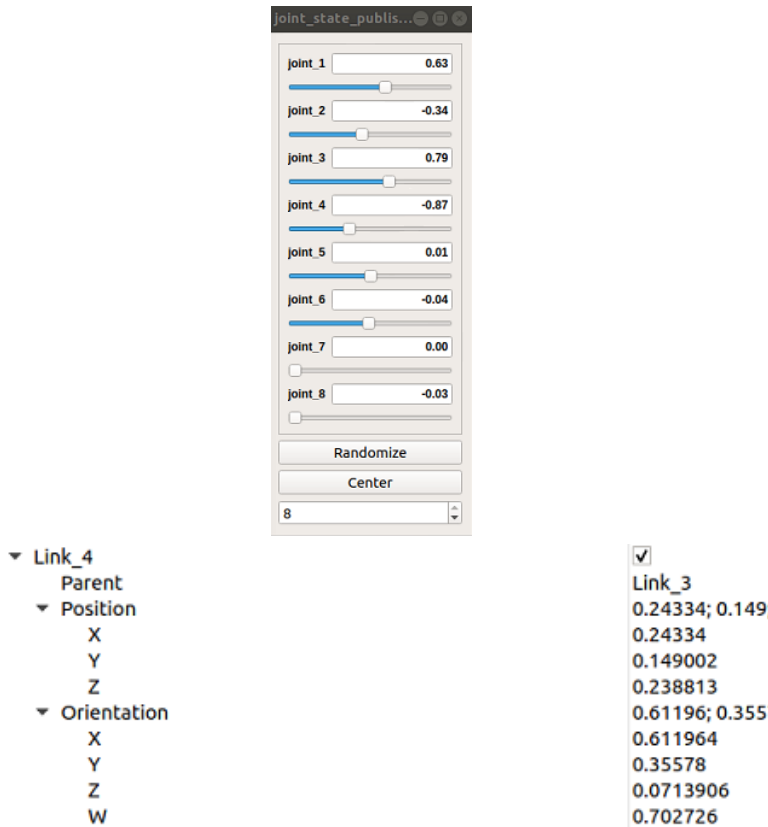


图 3-19 控制机械臂的位姿并查看其参数

② 打开源码文件，在源码文件中，根据 Rviz 中的位姿信息，修改对应的位姿参数，如下图示，可对位姿坐标进行修改。修改参数时也可以根据终端中的位姿信息进行修改，但是其精度没有 Rviz 中的高。代码修改完成后注意保存。


```

if __name__ == "__main__":
    moveItArm = MoveItArm()
    # 设置机械臂工作空间中的目标位姿, 位置使用x、y、z坐标描述,
    # 设置机械臂工作空间中的目标位姿
    target_pose = PoseStamped()
    target_pose.header.frame_id = moveItArm.reference_frame
    target_pose.header.stamp = rospy.Time.now()
    target_pose.pose.position.x = 0.24334
    target_pose.pose.position.y = 0.149002
    target_pose.pose.position.z = 0.238813
    target_pose.pose.orientation.x = 0.611964
    target_pose.pose.orientation.y = 0.35578
    target_pose.pose.orientation.z = 0.0713906
    target_pose.pose.orientation.w = 0.702726

    moveItArm.coordCtrl(target_pose) # 通过坐标控制机械臂

    # 关闭并退出moveit
    moveit_commander.roscpp_shutdown()
    moveit_commander.os._exit(0)

```

图 3-20 修改代码文件中的位姿参数

③ 在终端中运行机械臂控制指令，程序运行后即可观察到机械臂运动，同时 Rviz 和 GUI 中的参数信息也会同步变化。如果想要在终端中查看位姿数据，则在输入完运行指令后，同时在新终端中运行位姿数据查看命令，此时终端输出的位姿信息也会同步刷新显示。

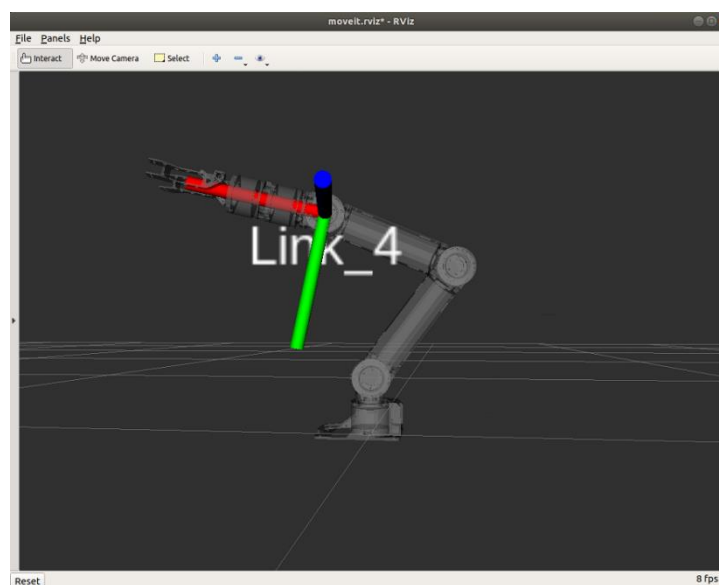


图 3-21 机械臂路径规划效果

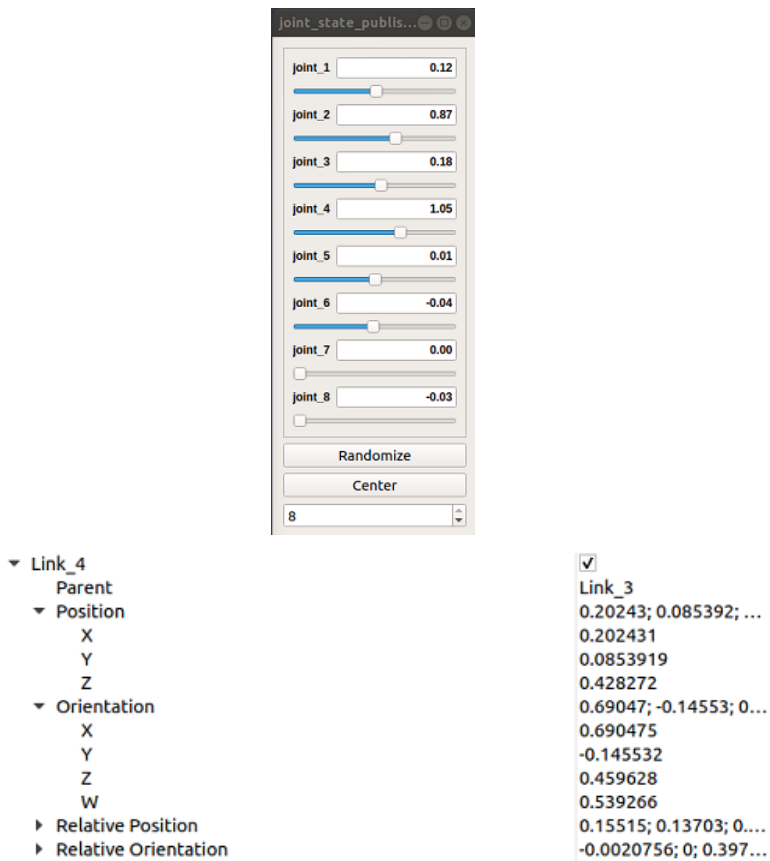


图 3-22 GUI 和 Rviz 中的位姿信息

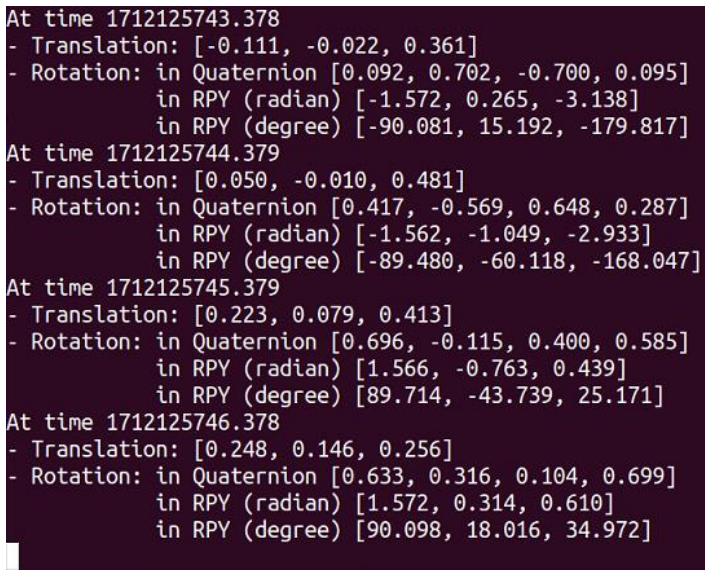


图 3-23 终端输出的机械臂位姿信息

(2) 多点路径规划

掌握自定义路径规划后，接下来是在主函数中通过设置新的位姿点并调用功能执行的函数实现机械臂在多个路径点间的连续规划。

最终需要实现的程序效果是：机械臂从当前位置运动到初始位置，然后运动到第一个指定位置，接着运动到第二个指定位置。

根据此效果在主函数中完成代码的补充，注意在设置新位置时先重新获取时间戳。

```

55 if __name__ == "__main__":
56     moveItArm = MoveItArm()
57     # 设置机械臂工作空间中的目标位姿，位置使用x、y、z坐标描述，
58     # 设置机械臂工作空间中的目标位姿
59     target_pose = PoseStamped()
60     target_pose.header.frame_id = moveItArm.reference_frame
61     target_pose.header.stamp = rospy.Time.now()
62     target_pose.pose.position.x = -0.0996125
63     target_pose.pose.position.y = -0.0204184
64     target_pose.pose.position.z = 0.109637
65     target_pose.pose.orientation.x = -0.297784
66     target_pose.pose.orientation.y = -0.642886
67     target_pose.pose.orientation.z = 0.643069
68     target_pose.pose.orientation.w = -0.290663
69
70     moveItArm.coordCtrl(target_pose) # 通过坐标控制机械臂
71
72
73
74     # 关闭并退出moveit
75     moveit_commander.roscpp_shutdown()
76     moveit_commander.os._exit(0)

```

图 3-24 待补充的主函数

4.拓展思考

如何改变机械臂的初始位姿并且实现指定重复次数的路径规划？

